

	Especialização em Inteligência Artificial Aplicada
	Projeto Final Integrado – Módulo 2 Alunos: Ivânio José da Rocha Júnior e Robson Pereira da Silva

Munka - Projeto de Pipeline de Dados em Nuvem e Aprendizagem de Máquina

Aprendizagem de Máquina — Prof. Dr. Sirlon Diniz

Cloud Computing — Prof. Dr. Raphael Gomes

Modelagem de Dados para IA — Prof. Dr. Otávio Calaça

Alunos: Ivânio José da Rocha Júnior e Robson Pereira da Silva

Sumário

1. Introdução	4
2. Contexto do Projeto	4
2.1. Definição do Problema	6
2.2. Domínio de Aplicação	6
2.3. Usuários e Tomadores de Decisão	6
2.4. Decisão Apoiada	7
2.5. Fontes de Dados	7
2.6. Tarefa de Aprendizagem de Máquina	9
3. Objetivo Geral	9
3.1. Objetivos Específicos	10
3.2. Resultado Esperado	11
3.3. Papel da Solução no Apoio à Decisão	11
4. Síntese da Proposta	13
5. Descrição da Solução	14
5.1. Arquitetura Geral da Solução	14
5.2. Arquitetura de Dados	16
5.3. Perfilamento, Volumetria e Ingestão dos Dados.....	18
5.4. Orquestração com Apache Airflow.....	21
5.5. Transformação dos Dados com dbt.....	24
5.6. Processamento das Evidências e Engenharia de Atributos	24
5.6.1. Parser de Evidências HTML/Textuais via RegEx no Snowflake	25
5.7. Qualidade e Governança dos Dados	26
6. Solução de Aprendizagem de Máquina	27
6.1. Preparação dos Dados, Prevenção de Data Leakage e Divisão (Data Splitting) ..	28
6.2. Otimização de Hiperparâmetros	31
6.3. Rastreamento e Registros no MLflow.....	33
6.4. Métricas de Avaliação e Resultados Comparativos	34
6.5. Análise Comparativa Detalhada: Scikit-Learn (Livre e Restrito) vs. NumPy.....	35
6.6. Diagnóstico dos Modelos por Curvas de Aprendizado e Resíduos	37
6.7. Análise da Importância das Features por Permutação	42
6.8. Aplicação Retrospectiva em Lote.....	45
7. Infraestrutura AWS	46

7.1. Componentes Utilizados	47
7.2. Mecanismo de Ingestão de Alta Performance (COPY INTO)	47
8. Arquitetura Equivalente Totalmente em AWS	48
8.1. Estimativa de Custos AWS & Snowflake	49
9. Visualização com Metabase.....	50
10. Reprodutibilidade da Solução	53
10.1. Avaliação do Pipeline	54
11. Limitações da Solução.....	55
12. Propostas de Evolução do Trabalho	55
12.1. Ampliação das Fontes de Dados	56
12.2. Integração com o GitLab.....	56
12.3. Integração com o Sistema de Helpdesk.....	58
12.4. Construção de uma Visão Integrada da Demanda	59
12.5. Novas Estratégias de Engenharia de Atributos	60
12.6. Evolução dos Modelos de Machine Learning	61
12.7. Processamento Semântico dos Textos	62
12.8. Monitoramento Contínuo e Aprendizado do Modelo	63
12.9. Governança, Segurança e Qualidade dos Dados	64
12.10. Impactos Esperados.....	65
13. Conclusão.....	66
14. Referências	67
ANEXO I: Implantação da Aplicação – Projeto Munka.	69

1. Introdução

O avanço da transformação digital na administração pública tem ampliado significativamente a quantidade de dados produzidos por sistemas corporativos. Esses dados, quando adequadamente organizados, tratados e analisados, podem ser utilizados não apenas para registrar fatos já ocorridos, mas também para apoiar processos de previsão e tomada de decisão.

Na Secretaria de Estado da Saúde de Goiás — SES-GO, diferentes sistemas de informação são utilizados para apoiar processos administrativos, operacionais e tecnológicos. Entre esses sistemas encontra-se o MUNKA, aplicação transacional empregada no registro, acompanhamento e controle de tarefas relacionadas à execução de serviços de Tecnologia da Informação.

Além de registrar informações sobre as atividades realizadas, o MUNKA participa do processo de medição dos serviços executados pela empresa contratada, atualmente a Mindsait, disponibilizando informações utilizadas para apuração dos valores relacionados às tarefas executadas.

Nesse contexto, o histórico acumulado pelo sistema representa uma fonte relevante de dados para aplicação de técnicas de Engenharia de Dados e Aprendizagem de Máquina.

A análise das tarefas já executadas pode permitir a identificação de padrões entre características das demandas, sua complexidade e o esforço efetivamente necessários para sua conclusão.

A proposta deste trabalho consiste, portanto, na construção de uma solução integrada de dados e Aprendizagem de Máquina capaz de utilizar o histórico do MUNKA para estimar retrospectivamente o esforço esperado das tarefas, considerando características estruturadas e atributos extraídos das evidências de execução. A estimativa produzida pelo modelo é comparada ao esforço efetivamente registrado, permitindo identificar desvios em relação aos padrões históricos observados.

A solução contempla a organização e preparação dos dados, construção de um pipeline de processamento, transformação dos dados em uma estrutura analítica, aplicação de técnicas de Aprendizagem de Máquina e disponibilização dos resultados para apoio à tomada de decisão.

2. Contexto do Projeto

O MUNKA é um sistema transacional utilizado pela Secretaria de Estado da Saúde de Goiás para registro, acompanhamento e controle de tarefas executadas no contexto da prestação de serviços de desenvolvimento, manutenção e sustentação de sistemas de informação.

O sistema armazena dados referentes às tarefas realizadas, permitindo registrar informações relacionadas às atividades, projetos, responsáveis, características das demandas, evidências de execução e demais elementos utilizados no acompanhamento dos serviços.

Além de sua função operacional, o MUNKA é utilizado como parte do processo de medição dos serviços executados pela empresa contratada. A partir das informações registradas no sistema, são realizados procedimentos que contribuem para a apuração dos valores relacionados às tarefas executadas e, consequentemente, para o processo de pagamento da contratada.

Esse cenário apresenta uma característica predominantemente retrospectiva: após a execução da tarefa, as informações são registradas, consolidadas e utilizadas no processo de medição.

Para fins de gestão e acompanhamento contratual, existe a necessidade de dispor de referências quantitativas que auxiliem a análise do esforço registrado e permitam identificar tarefas que apresentem comportamento diferente do padrão histórico, viabilizando a possibilidade de redução do número de tarefas a serem revisadas.

Em média, o volume se aproxima de 5.000 tarefas mensais. A equipe que executa os procedimentos de revisão é reduzida em vista do número de tarefas, justificando a construção de ferramenta que auxilie na verificação das horas executadas.

Nesta primeira versão, essa referência é construída de forma retrospectiva: o modelo utiliza informações consolidadas da tarefa e de suas evidências para estimar o esforço esperado e compará-lo com as horas efetivamente registradas.

O histórico de tarefas existentes no MUNKA constitui a base para essa estimativa retrospectiva. As tarefas concluídas possuem características que podem ser relacionadas ao esforço necessário para sua execução, possibilitando a aplicação de técnicas de Aprendizagem de Máquina para identificação de padrões históricos.

Além dos dados estruturados existentes nas tabelas transacionais, o MUNKA também possui informações de natureza não estruturada ou semiestruturada, especialmente nos campos textuais e nas evidências associadas às tarefas.

Essas informações podem ser processadas para obtenção de atributos adicionais, ampliando a capacidade de caracterização das atividades e enriquecendo o conjunto de dados utilizado durante o treinamento dos modelos.

A proposta do projeto é transformar esse conjunto de informações transacionais em uma estrutura analítica que sustente o modelo retrospectivo atual, ferramentas de visualização e, futuramente, modelos preditivos antecipados construídos apenas com informações disponíveis antes do início da execução.

Dessa forma, o projeto amplia o uso dos dados do MUNKA de uma abordagem exclusivamente descritiva para uma abordagem analítica baseada em estimativas retrospectivas e comparação de desvios. A previsão antecipada de novas tarefas é tratada como evolução futura da solução.

2.1. Definição do Problema

O problema tratado neste trabalho pertence ao domínio de gestão e acompanhamento de serviços de Tecnologia da Informação da Secretaria de Estado da Saúde de Goiás.

O problema central deste trabalho consiste em utilizar os dados históricos das tarefas registradas no MUNKA, incluindo suas características estruturadas e informações extraídas das evidências de execução, para construir um modelo capaz de estimar retrospectivamente o esforço esperado de uma tarefa e compará-lo ao esforço efetivamente registrado.

A estimativa gerada pelo modelo poderá ser comparada à quantidade de horas efetivamente registrada, permitindo identificar situações em que o esforço observado apresente comportamento significativamente diferente daquele esperado segundo os padrões históricos aprendidos.

Dessa forma, a solução possui caráter retrospectivo e analítico, funcionando como instrumento adicional de apoio ao acompanhamento e à análise das medições realizadas.

2.2. Domínio de Aplicação

O domínio de aplicação do projeto corresponde à gestão de serviços de Tecnologia da Informação no âmbito da Secretaria de Estado da Saúde de Goiás, especialmente no contexto de acompanhamento, execução e medição de tarefas realizadas pela empresa contratada.

O projeto utiliza dados originados de um sistema corporativo real, empregado no acompanhamento das atividades realizadas no ambiente institucional.

A solução está direcionada ao apoio à gestão das tarefas, permitindo transformar registros operacionais em informações analíticas e estimativas retrospectivas utilizadas na identificação de desvios.

2.3. Usuários e Tomadores de Decisão

Os principais usuários da solução são os gestores e profissionais responsáveis pelo acompanhamento das demandas de Tecnologia da Informação da SES-GO, bem como aqueles envolvidos nos processos de análise, acompanhamento e medição dos serviços executados pela empresa contratada.

A solução poderá oferecer informações adicionais para profissionais responsáveis por:

- acompanhamento das tarefas;

- avaliação das demandas;
- análise de complexidade;
- planejamento das atividades;
- acompanhamento do esforço empregado;
- medição dos serviços;
- avaliação de diferenças entre valores previstos e realizados.

O modelo não substitui a análise técnica ou administrativa realizada pelos responsáveis pela gestão contratual.

Seu objetivo é fornecer uma referência adicional baseada no comportamento histórico dos dados.

2.4. Decisão Apoiada

A principal decisão apoiada pela solução consiste na identificação de tarefas cujo esforço efetivamente registrado apresente diferença relevante em relação ao esforço esperado segundo o comportamento histórico das atividades existentes no MUNKA.

A partir das características da tarefa e das evidências produzidas durante sua execução, o modelo gera uma estimativa de horas que pode ser utilizada como referência independente para comparação com as horas registradas.

A solução poderá apoiar questões como:

Quais tarefas apresentam as maiores diferenças entre esforço estimado e realizado?

Qual seria o esforço esperado para esta tarefa considerando tarefas historicamente semelhantes?

O número de horas registrado está próximo do valor estimado pelo modelo?

Existem determinados tipos de tarefa que apresentam sistematicamente maior variação?

Quais características estão mais relacionadas ao esforço necessário para execução?

2.5. Fontes de Dados

A principal fonte de dados utilizada neste trabalho é o banco de dados transacional do sistema MUNKA.

Os dados históricos registrados pelo sistema representam as tarefas executadas no contexto dos serviços de Tecnologia da Informação acompanhados pela SES-GO.

O projeto utiliza duas categorias principais de dados.

Dados estruturados

Os dados estruturados são provenientes das entidades e tabelas do sistema MUNKA relacionadas às tarefas e aos demais elementos necessários para caracterizá-las.

Essas informações podem incluir atributos associados a:

- identificação da tarefa;
- projeto;
- responsáveis;
- características da atividade;
- status;
- datas;
- complexidade;
- esforço registrado;
- informações utilizadas na medição;
- demais relacionamentos disponíveis no modelo transacional.

A definição exata das tabelas, atributos e relacionamentos utilizados será apresentada na seção específica de descrição e modelagem dos dados.

Dados não estruturados ou semiestruturados

O projeto também considera informações textuais e evidências registradas no sistema, como os dados inseridos no campo de evidências de cada tarefa, que recebe formato em texto, códigos e imagens ou até o campo de indicação dos links para o gitlab.

Esses dados podem conter descrições sobre as tarefas e elementos utilizados para demonstrar ou documentar sua execução.

O processamento dessas informações permitirá extrair atributos adicionais que poderão ser utilizados na etapa analítica e, quando aplicável, no treinamento dos modelos de Aprendizagem de Máquina.

A utilização conjunta dos dados estruturados e das informações textuais atende ao objetivo de construir um conjunto de dados mais representativo das características das tarefas.

2.6. Tarefa de Aprendizagem de Máquina

Do ponto de vista de Aprendizagem de Máquina, o problema é caracterizado como uma tarefa de **regressão supervisionada**.

A variável-alvo corresponde ao esforço necessário para execução de uma tarefa, representado em horas.

As características históricas das tarefas são utilizadas como variáveis explicativas.

De forma simplificada, o processo pode ser representado como:

Características da tarefa + evidências de execução → Modelo de Aprendizagem de Máquina
→ Horas estimadas retrospectivamente

A aplicação retrospectiva segue pela comparação:

Horas estimadas pelo modelo ↔ Horas executadas → Análise de desvios

O treinamento utiliza tarefas históricas para que o modelo aprenda relações entre as características observadas e o esforço realizado.

Após o treinamento, o modelo é aplicado a tarefas já executadas e caracterizadas por seus dados estruturados e evidências, produzindo uma estimativa retrospectiva de esforço para comparação com as horas registradas.

O projeto avalia o desempenho dessa estimativa utilizando métricas adequadas para problemas de regressão e compara os resultados obtidos entre a implementação manual do algoritmo e a implementação utilizando bibliotecas Python.

3. Objetivo Geral

O objetivo geral deste trabalho é desenvolver uma solução integrada de Engenharia de Dados e Aprendizagem de Máquina capaz de utilizar os dados históricos do sistema MUNKA, incluindo características estruturadas das tarefas e atributos extraídos das evidências de execução, para estimar o esforço esperado, em horas, associado às atividades realizadas e permitir sua comparação com o esforço efetivamente registrado.

A solução busca fornecer aos gestores uma referência analítica adicional para acompanhamento das tarefas, identificação de desvios e análise dos padrões de esforço observados historicamente.

3.1. Objetivos Específicos

Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos para a primeira versão:

- Identificar tarefas com maiores desvios entre valores estimados e realizados, indicando necessidade de revisão das mesmas;
- Identificar as fontes de dados do MUNKA relevantes para caracterização das tarefas;
- Selecionar atributos históricos que apresentem potencial relação com o esforço necessário para execução das atividades;
- Organizar e documentar os dados estruturados utilizados no projeto;
- Identificar e processar informações não estruturadas ou semiestruturadas associadas às tarefas;
- Realizar limpeza, padronização e tratamento dos dados;
- Extrair atributos que possam melhorar a representação das características das tarefas;
- Construir um pipeline de dados reproduzível;
- Armazenar os dados brutos e processados em ambiente de nuvem;
- Utilizar o Snowflake como base analítica para organização dos dados tratados;
- Utilizar o dbt para transformação, documentação e testes dos modelos analíticos;
- Utilizar o Airflow para orquestração das etapas do pipeline;
- Estruturar modelos de dados para staging, dimensões, fatos e bases destinadas ao consumo analítico;
- Construir um conjunto de dados adequado ao treinamento do modelo;
- Implementar uma técnica de Aprendizagem de Máquina de forma manual, em hard-code;
- Implementar a mesma técnica utilizando bibliotecas Python;
- Comparar os resultados obtidos pelas duas abordagens;
- Construir um baseline para comparação do desempenho dos modelos;
- Avaliar os modelos utilizando métricas adequadas para problemas de regressão;
- Registrar as estimativas produzidas pelo modelo;
- Utilizar as horas estimadas pelo modelo como referência para análise das medições realizadas;

- Disponibilizar informações analíticas e resultados do modelo em um dashboard;
- Comparar o esforço estimado pelo modelo com o esforço efetivamente registrado nas tarefas;
- Identificar limitações existentes nos dados e no modelo;
- Analisar situações de acerto e erro das estimativas;
- Documentar a arquitetura, o pipeline, os resultados, as limitações e as possibilidades de evolução da solução.

3.2. Resultado Esperado

O principal resultado esperado é a construção de um modelo retrospectivo capaz de comparar o esforço esperado com o efetivamente executado para tarefas já caracterizadas por seus dados estruturados e pelas evidências produzidas durante sua execução, visando dados que indiquem a necessidade de revisão das tarefas no momento de fiscalização das mesmas.

A estimativa será comparada às horas efetivamente registradas, permitindo analisar a aderência das tarefas aos padrões históricos identificados pelo modelo.

Nesta primeira versão, a solução tem caráter predominantemente analítico e retrospectivo. A previsão antecipada de esforço para novas tarefas, utilizando exclusivamente informações disponíveis antes do início da execução, é apresentada como uma evolução futura.

3.3. Papel da Solução no Apoio à Decisão

A solução proposta possui caráter de apoio à decisão. As retrospectivas produzidas pelo modelo não devem ser interpretadas como valores definitivos nem como substituição das regras formais de medição contratual.

O modelo trabalha com padrões aprendidos a partir de dados históricos e, portanto, está sujeito às características, limitações e possíveis distorções existentes nesses dados. Seu resultado deverá ser utilizado como uma referência adicional para análise.

Uma estimativa retrospectiva significativamente diferente do esforço efetivamente registrado não representa, isoladamente, evidência de erro na execução ou na medição da tarefa.

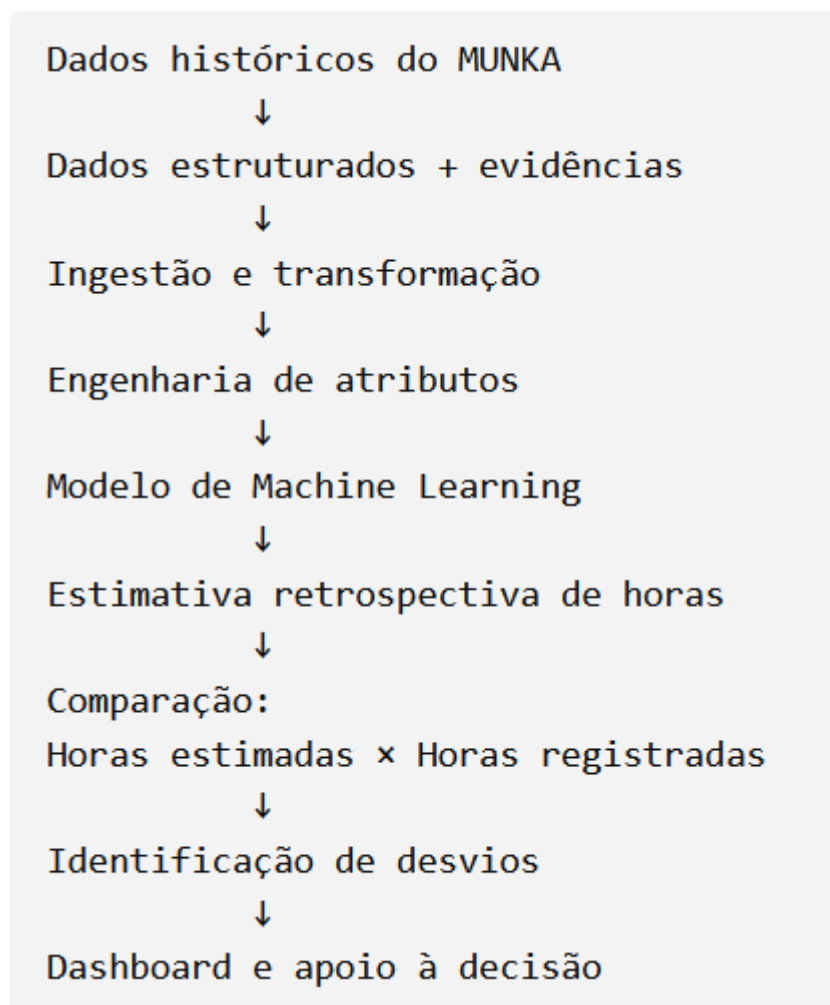
Entretanto, essa diferença poderá indicar casos que mereçam análise mais detalhada por via de revisão.

 INSTITUTO FEDERAL Goiás	Especialização em Inteligência Artificial Aplicada
	Projeto Final Integrado – Módulo 2 Alunos: Ivânio José da Rocha Júnior e Robson Pereira da Silva

A aplicação de Aprendizagem de Máquina ao histórico do MUNKA pretende, dessa forma, disponibilizar à gestão um instrumento adicional de acompanhamento, análise das medições e identificação de situações que mereçam avaliação mais detalhada.

4. Síntese da Proposta

A proposta pode ser resumida no seguinte fluxo conceitual:



A construção desse fluxo permite integrar os conhecimentos de Modelagem de Dados para IA, Aprendizagem de Máquina e Cloud Computing, conforme estabelecido na proposta do Projeto Final Integrado do Módulo 2 da Pós-Graduação em Inteligência Artificial Aplicada.

5. Descrição da Solução

5.1. Arquitetura Geral da Solução

A solução desenvolvida para o projeto MUNKA foi estruturada como uma plataforma integrada de Engenharia de Dados e Aprendizagem de Máquina, com o objetivo de transformar os dados transacionais das tarefas em uma base analítica adequada ao treinamento e à aplicação de modelos de Machine Learning, incluindo o modelo retrospectivo adotado nesta primeira versão.

A arquitetura adotada estabelece um fluxo de dados desde a extração dos registros do MUNKA até a disponibilização de informações tratadas para análise e treinamento dos modelos de Machine Learning. Para isso, foram integradas tecnologias de armazenamento em nuvem, Data Warehouse, transformação de dados, orquestração, processamento analítico e rastreamento de experimentos.

Os principais componentes utilizados são:

- Amazon Simple Storage Service (S3) e AWS Identity and Access Management (IAM), para armazenamento dos arquivos de dados de origem, com volume de 2,1 GiB de dados até este momento e usuário IAM com política de leitura em tempo de execução (Least Privilege Principle);
- Snowflake, como plataforma de armazenamento e processamento analítico;
- dbt Core 1.5, para transformação, organização, testes e documentação dos dados;
- Apache Airflow 2.x, para orquestração das diferentes etapas do pipeline. Sua execução ocorre em contêiner Docker (docker-compose), coordenando a DAG Master `dag\_munka\_full\_pipeline` e suas 7 DAGs especializadas (APACHE SOFTWARE FOUNDATION, s.d.);
- Docker, para execução e padronização do ambiente do Airflow e de outros serviços;
- Python e NumPy, para implementação manual do algoritmo de Machine Learning;
- Scikit-Learn, para implementação equivalente do modelo utilizando biblioteca especializada;
- MLflow, para rastreamento dos experimentos de Machine Learning;
- Optuna, para experimentação com otimização de hiperparâmetros;
- Metabase, utilizado como camada de visualização e consumo dos dados armazenados no Snowflake;
- AWS CloudFormation, utilizado na definição da infraestrutura AWS como código.

A solução foi implementada utilizando uma abordagem de container, favorecendo a reprodutibilidade e o isolamento dos componentes, assim como facilidade de implantação local conectada aos serviços em nuvem.

Foi adotada a centralização das variáveis de configuração em uma fonte única (Single Source of Truth — SSOT), utilizando arquivo `.env` fora do controle de versão e injeção dinâmica das configurações nos contêineres Airflow, dbt e Python.

De forma simplificada, o fluxo implementado pode ser representado da seguinte maneira:

MUNKA → arquivos CSV → Amazon S3 → Snowflake RAW → dbt STAGING → dbt INTERMEDIATE → GOLD / ML → Machine Learning e visualização

Essa organização tem como objetivo permitir rastreabilidade entre o dado originalmente extraído do sistema transacional e as informações posteriormente utilizadas pelos modelos analíticos e pela análise retrospectiva.

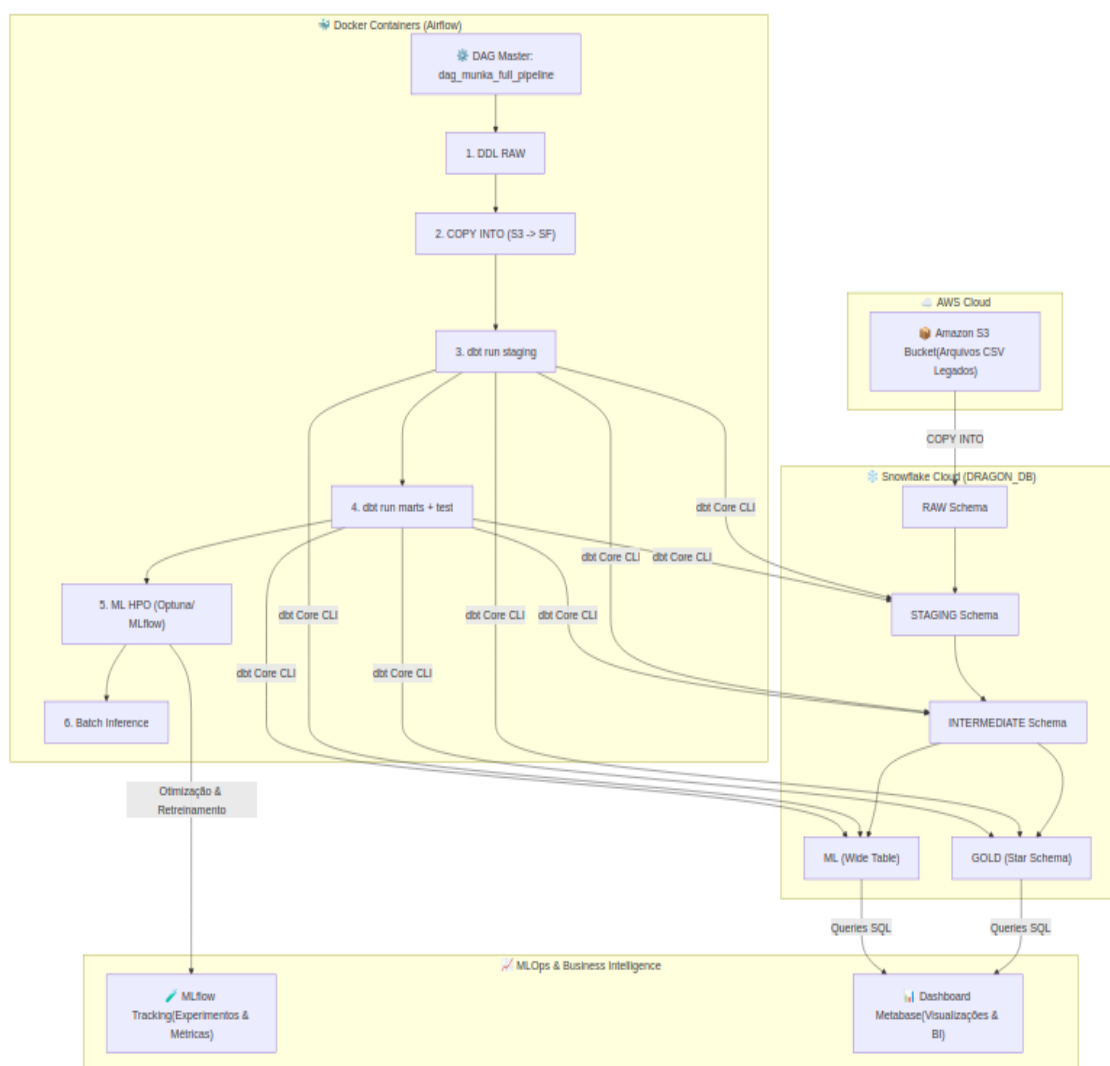


Figura 1. Arquitetura Geral da Solução.

5.2. Arquitetura de Dados

A arquitetura de dados foi organizada segundo os princípios de uma Arquitetura Medallion, adaptada às necessidades do projeto.

Foram definidos os seguintes schemas no Snowflake:

- MUNKA_RAW;
- MUNKA_STG;
- MUNKA_INT;
- MUNKA_GOLD;
- MUNKA_ML.

Cada camada possui uma responsabilidade específica dentro do fluxo de processamento. Abaixo a arquitetura de dados:

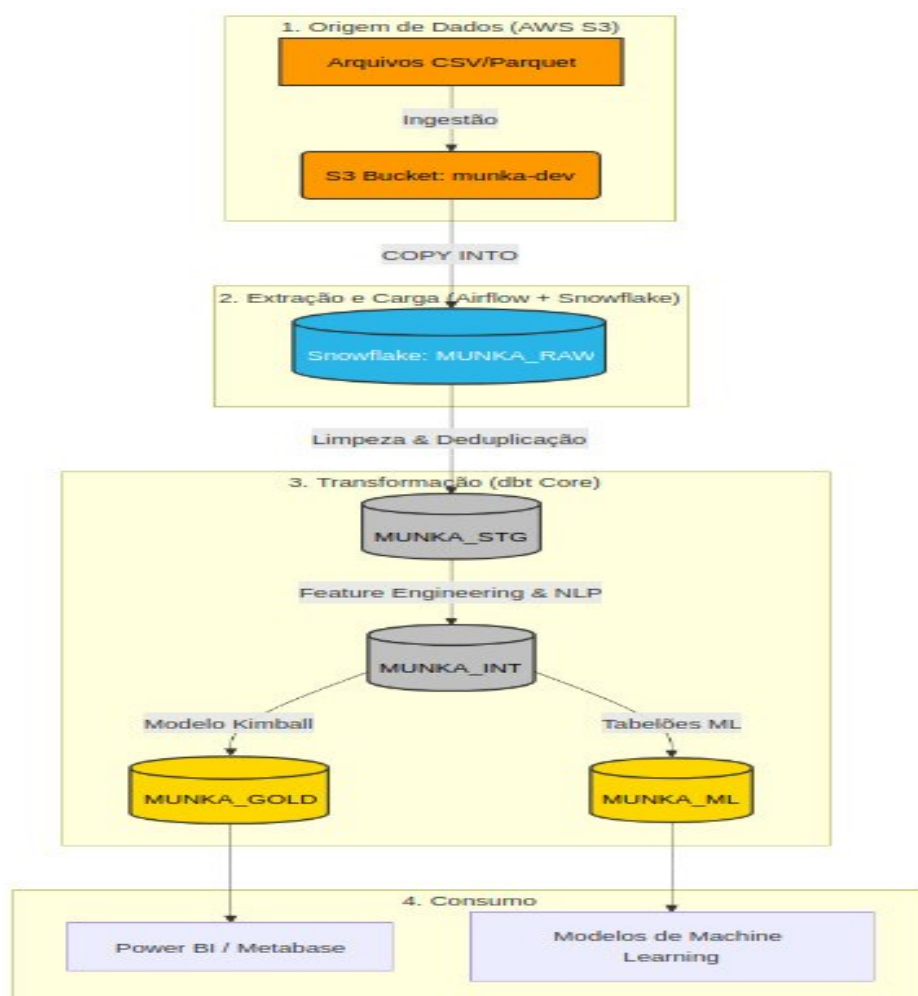


Figura 2. Arquitetura de Dados.

5.2.1. Camada RAW

A camada MUNKA_RAW representa o primeiro nível de persistência dos dados no Snowflake.

Os dados provenientes do ambiente transacional do MUNKA são exportados para arquivos CSV e armazenados inicialmente no Amazon S3. A partir desses arquivos, o pipeline realiza a carga para tabelas correspondentes na camada RAW do Snowflake.

Foram previstas **47 tabelas RAW**, refletindo as estruturas de origem necessárias ao projeto.

A criação dessas tabelas é realizada por meio de macros dbt acionadas pelo Apache Airflow. A estratégia utiliza comandos CREATE TABLE IF NOT EXISTS, permitindo que a criação das estruturas seja idempotente, isto é, possa ser executada novamente sem substituir indevidamente tabelas que já contenham dados.

Nessa etapa, procura-se manter os dados o mais próximo possível de sua representação original, deixando as regras de limpeza e transformação para as camadas posteriores.

5.2.2. Staging

A camada MUNKA_STG realiza as primeiras transformações sobre os dados carregados na RAW.

Nessa camada são aplicadas operações de:

- limpeza;
- padronização;
- tipagem;
- tratamento das estruturas provenientes da origem;
- deduplicação de registros.

Foi feita a utilização de QUALIFY ROW_NUMBER() como uma das estratégias empregadas para identificação e remoção de duplicidades com base nas chaves e critérios temporais definidos nos modelos.

A Staging procura preservar a semântica das informações da origem, evitando concentrar regras analíticas complexas nesta etapa.

5.2.3. Intermediate

A camada MUNKA_INT concentra transformações que dependem de regras mais elaboradas e que não pertencem diretamente nem à Staging nem às tabelas dimensionais finais.

Essa camada possui especial importância no tratamento das informações relacionadas às evidências das tarefas.

É nela que foi implementado o modelo: `int_tarefa_evidencias_features.sql`, responsável por extrair atributos derivados dos conteúdos existentes nas evidências.

5.2.4. Gold

A camada MUNKA_GOLD representa a estrutura analítica destinada principalmente ao consumo por ferramentas de Business Intelligence.

Ela utiliza uma modelagem dimensional baseada no modelo estrela, com tabelas de fatos e dimensões.

Entre os exemplos documentados encontram-se dimensões relacionadas a projetos, sprints e usuários e fatos relacionados às tarefas, evidências e faturamento.

Foram documentadas aproximadamente **39 tabelas dimensionais e fatos** nessa camada.

Para identificação das entidades analíticas, foram utilizadas **Surrogate Keys**, geradas com funções de HASH a partir das respectivas chaves naturais.

Essa estratégia desacopla as chaves utilizadas no modelo analítico dos identificadores existentes nas tabelas transacionais.

5.2.5. Camada de Machine Learning

Além da estrutura dimensional, foi criada uma camada específica denominada MUNKA_ML. Seu principal componente documentado é: `ml_tarefa_features`

Trata-se de uma estrutura desnormalizada do tipo *Wide Table*, destinada ao consumo pelos processos de Machine Learning.

A finalidade dessa tabela é reunir previamente as informações necessárias ao treinamento, evitando que o processo de Ciência de Dados precise reconstruir repetidamente relacionamentos entre diversas tabelas utilizando operações de JOIN em Python.

Dessa forma, parte significativa da preparação dos dados é executada no próprio Snowflake, antes que o conjunto seja entregue aos algoritmos de Machine Learning.

5.3. Perfilamento, Volumetria e Ingestão dos Dados

O conjunto de dados utilizado no projeto compreende o período de **15 de janeiro de 2023 a 30 de junho de 2026**, abrangendo o histórico de execuções de tarefas e sprints registrado no sistema MUNKA.

Na etapa inicial de ingestão, foram carregados **15.420 registros brutos** na camada MUNKA_RAW.RAW_TAREFA, preservando-se os dados originais para rastreabilidade e posterior tratamento.

Durante o processo de preparação dos dados, foram aplicadas regras de limpeza, consistência e seleção das observações adequadas ao treinamento dos modelos de Machine Learning. Ao final desse processo, **5.000 registros**, correspondentes a aproximadamente **32,4% do volume originalmente ingerido**, foram considerados válidos

e disponibilizados na camada `MUNKA_ML.ML_TAREFA_FEATURES`. Os **10.420 registros restantes**, equivalentes a aproximadamente **67,6% da base bruta**, foram descartados em razão de critérios como **deduplicação por meio da função `ROW_NUMBER()`**, exclusão de tarefas canceladas e remoção de registros sem apontamento de horas, uma vez que estes não forneciam informação suficiente para a variável utilizada como referência na modelagem.

Após o tratamento, o dataset utilizado pelo modelo apresentou **0% de valores ausentes (missing values)**. Os campos numéricos com ausência de informação foram tratados por meio de imputação com valor zero utilizando `df.fillna(0)`, complementada, quando aplicável, pela criação de **flags binárias** destinadas a indicar a ocorrência original de valores ausentes. Dessa forma, buscou-se preservar essa informação sem comprometer a execução dos algoritmos.

A análise estatística também identificou 142 observações classificadas como outliers, utilizando como referência o critério do Intervalo Interquartil (IQR), com limite de $1,5 \times \text{IQR}$, além de 18 ocorrências consideradas extremas, caracterizadas por tarefas com duração superior a 100 horas. Esses registros foram mantidos para preservar a representatividade do histórico. O pipeline utiliza padronização com `StandardScaler` e, quando aplicável, regularização L2 (parâmetro α), porém essas técnicas não constituem tratamento robusto específico de outliers. A análise de resíduos apresentada no Capítulo 6 mostra que tarefas de maior duração continuam representando maior dificuldade de estimação.

Como limitação do conjunto de dados, destaca-se a alta variabilidade observada em tipos de manutenção com baixa frequência histórica, o que reduz a quantidade de exemplos disponíveis para o aprendizado desses padrões específicos. Adicionalmente, a qualidade das estimativas depende diretamente da precisão dos apontamentos de horas realizados pelos usuários do sistema, uma vez que inconsistências, atrasos ou erros no registro das horas executadas podem introduzir ruído na variável utilizada como referência para treinamento e avaliação dos modelos. Resumo Técnico:

Dimensão de análise	Valor / detalhamento	Observações técnicas
Período dos dados	15/01/2023 a 30/06/2026	Cobertura histórica das tarefas e evidências do sistema MUNKA.
Quantidade total de tarefas	15.420 registros	Total de tarefas ingeridas na camada MUNKA_RAW.RAW_TAREFA.
Quantidade utilizada no ML	5.000 registros (32,4%)	Amostras consolidadas e preparadas na camada MUNKA_ML.ML_TAREFA_FEATURES.
Quantidade descartada	10.420 registros (67,6%)	Deduplicações, tarefas canceladas e registros sem apontamento.
Valores ausentes	0% na camada ML	Imputação com zero nos atributos em que a ausência foi tratada dessa forma no pipeline.
Outliers	142 pelo IQR / 18 > 100 h	Registros mantidos. StandardScaler e L2 foram usados no treinamento, mas não constituem tratamento robusto específico de outliers.
Limitações	Baixa amostragem em categorias raras	Alta variância em tipos raros de manutenção e dependência da qualidade do registro humano.

Tabela 1. Perfilamento e Volumetria dos Dados.

A ingestão dos dados foi estruturada utilizando o Amazon S3 como área de armazenamento dos arquivos CSV exportados do MUNKA.

O processo é executado pelo Apache Airflow por meio da DAG passo2_s3_to_snowflake_munka_raw.

A responsabilidade dessa DAG é realizar a carga dos arquivos existentes no S3 para as respectivas tabelas da camada MUNKA_RAW no Snowflake. O carregamento utiliza comandos COPY INTO, executados no Snowflake, permitindo que os arquivos armazenados no S3 sejam carregados diretamente para o Data Warehouse.

Assim, o fluxo de ingestão é constituído pelas seguintes etapas:

Camada	Schema Snowflake	Descrição e Papel na Arquitetura
Bronze (RAW)	MUNKA_RAW	Ingestão bruta idêntica à fonte S3 via comando <code>COPY INTO</code> . Sem transformações de tipo ou regras de negócio.
Silver (STAGING)	MUNKA_STG	Limpeza, padronização de nomenclatura (snake_case), tipagem forte de dados e deduplicação inteligente usando <code>QUALIFY ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC)</code> .
Intermediate	MUNKA_INT	Transformação intermediária com extração de <i>features</i> textuais/HTML via Expressões Regulares (Regex) para engenharia de atributos.
Gold (MARTS)	MUNKA_GOLD	Modelagem Dimensional Estrela (<i>Star Schema</i>) com 25+ Dimensões (<code>dim_*</code>) e 10+ Tabelas Fato (<code>fact_*</code>) e 78 testes de qualidade automatizados.
Gold (ML)	MUNKA_ML	Tabelão Desnormalizado (<i>Wide Table</i>) <code>ml_tarefa_features</code> agregando dados de tarefas, histórico, projeto e evidências extraídas para alimentar os modelos de Machine Learning.

Tabela 2. Denominação e Papéis das Camadas de Dados.

Na implementação atual, a DAG de ingestão gera comandos COPY INTO com credenciais IAM obtidas dinamicamente da conexão AWS configurada no Airflow, sem valores fixos no

código SQL. Como evolução de segurança, pode-se substituir esse mecanismo por Snowflake Storage Integration associada a uma IAM Role, eliminando a necessidade de chaves de acesso na execução.

Abaixo as camadas criadas no Snowflake:

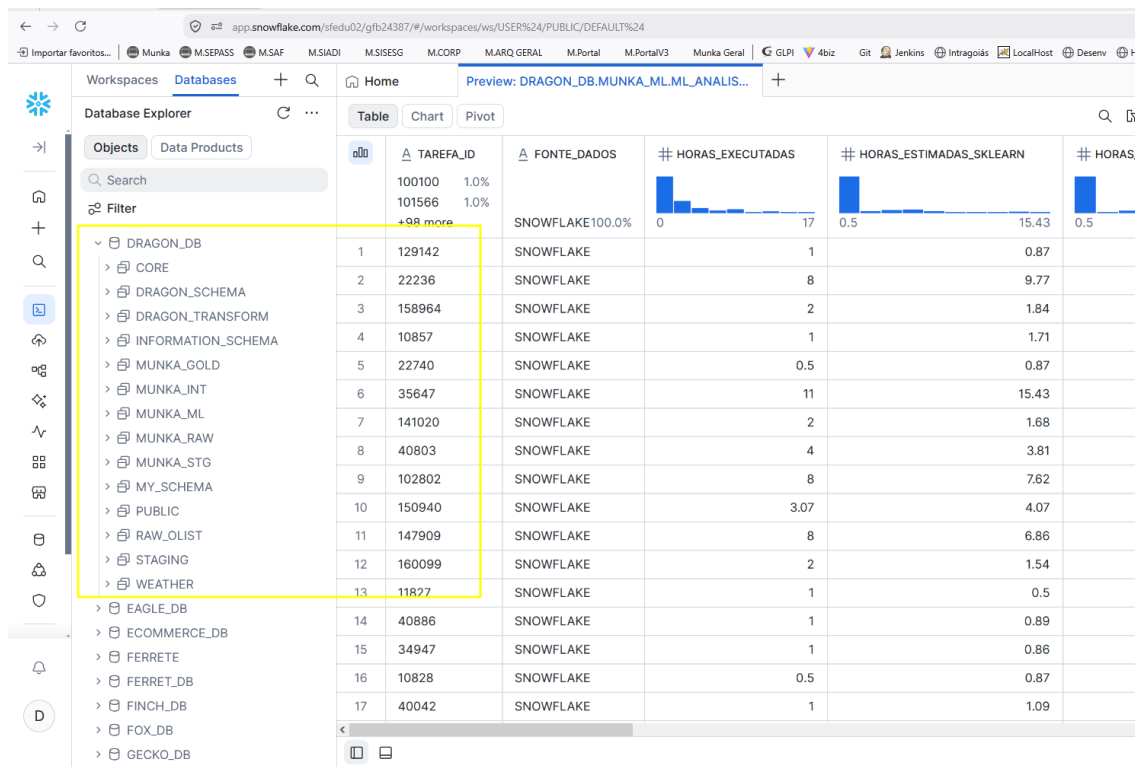


Figura 3. Bases MUNKAS criadas no Snowflake do esquema DRAGON_DB.

5.4. Orquestração com Apache Airflow

Para a primeira versão, o Apache Airflow ficou responsável pela coordenação das diferentes etapas do pipeline. O ambiente de execução foi estruturado com Docker, permitindo que as dependências necessárias à execução do Airflow e do dbt sejam instaladas e disponibilizadas de maneira padronizada.

O processamento foi dividido nas seguintes DAGs (**Directed Acyclic Graph**):

Passo 1 — DDL RAW (passo1_munka_dbt_create_raw_tables)

- **Objetivo:** Criar a estrutura das tabelas na camada MUNKAS_RAW.
- **Tecnologia:** dbt Macros (run-operation create_raw_tables).

	Especialização em Inteligência Artificial Aplicada
	Projeto Final Integrado – Módulo 2 Alunos: Ivânio José da Rocha Júnior e Robson Pereira da Silva

Passo 2 — Carga S3 para Snowflake (passo2_s3_to_snowflake_munka_raw)

- **Objetivo:** Ingerir os dados dos 47 arquivos CSV do S3 diretamente para o Snowflake.
- **Tecnologia:** Airflow SnowflakeOperator executando COPY INTO MUNKA_RAW.<tabela> FROM @MUNKA_RAW.S3_STAGE usando credenciais IAM dinâmicas.

Passo 3 — Transformação Staging (passo3_munka_dbt_create_stg)

- **Objetivo:** Padronizar, tratar nulos e deduplicar registros.
- **Comando dbt:** dbt run --select staging.

Passo 4 — Modelagem Marts e Qualidade de Dados (passo4_munka_dbt_run_marts)

- **Objetivo:** Construir o Star Schema (Gold) e executar a suíte de testes de qualidade.
- **Comando dbt:** dbt run --select intermediate marts seguido de dbt test.
- **Garantia de Qualidade: 78 testes de integridade automatizados** (unique e not_null) aprovados com 100% de sucesso.

Passo 5 — Treinamento de ML e HPO (passo5_ml_hpo_e_retreinamento)

- **Objetivo:** Buscar os melhores hiperparâmetros (Optuna), retrainar os modelos MLP (Scikit-Learn e NumPy) com 5-Fold Cross Validation e registrar artefatos e métricas usando o MLflow.
- **Saída:** Arquivos sklearn_best_params.json, numpy_best_params.json e experimentos registrados no MLflow.

Passo 6 — Análise Retrospectiva em Lote (passo6_batch_inference)

- **Objetivo:** Carregar os modelos retreinados e o escalador para executar a análise retrospectiva sobre tarefas já concluídas (HORAS_EXECUTADAS IS NOT NULL), viabilizando a comparação entre horas estimadas e horas registradas.
- **Saída:** arquivo analise_retrospectiva.csv, com horas executadas, estimativas dos modelos, erros absolutos, erros percentuais, diferença entre modelos e metadados da análise; adicionalmente, analise_retrospectiva_metrics.json registra as métricas agregadas da execução.

Passo 7 — Carga e Modelagem da Análise Retrospectiva

- **Objetivo:** Sincronizar o arquivo produzido pela análise retrospectiva em lote (`analise\_retrospectiva.csv`), carregá-lo via dbt seed e materializar a mart analítica `ml\_analise\_retrospectiva` no schema MUNKA_ML do Snowflake.

- Comando dbt: dbt seed --select analise_retrospectiva seguido de dbt run --select ml_analise_retrospectiva.
- Saída: Tabela analítica MUNKA_ML.ML_ANALISE_RETROSPECTIVA disponível para exploração e visualização em dashboards no Metabase.

Durante o desenvolvimento do projeto, também foi criada uma DAG de nível superior denominada: dag_munka_full_pipeline, com o objetivo de coordenar sequencialmente as etapas do processamento. Conceitualmente, essa orquestração estabelece o seguinte fluxo:

preparação da RAW → ingestão S3/Snowflake → Staging → Intermediate/Marts → testes dbt → treinamento de Machine Learning → Análise retrospectiva em lote

Com essa organização, o Airflow assume a responsabilidade de controlar as dependências entre as etapas, enquanto as transformações propriamente ditas permanecem concentradas no dbt e no Snowflake.

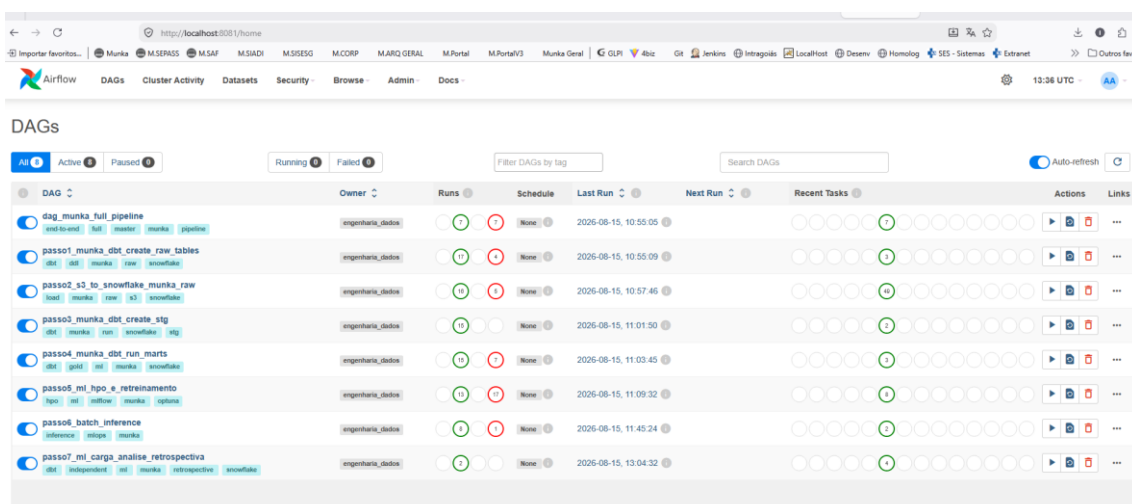


Figura 4. Estrutura do pipeline do projeto no Airflow.

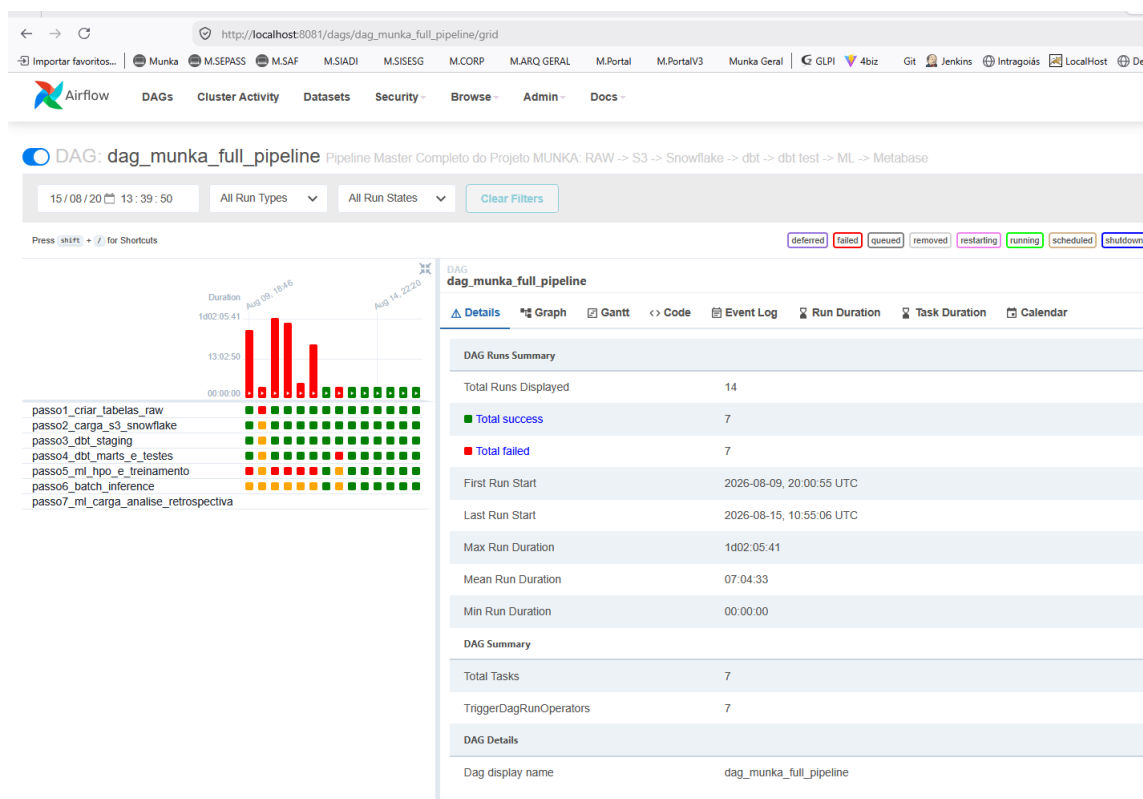


Figura 5. Execuções do pipeline principal da aplicação.

5.5. Transformação dos Dados com dbt

O dbt Core é utilizado como principal componente da etapa de transformação do pipeline ELT. (DBT LABS, s.d.)

A estrutura anterior, baseada em arquivos SQL monolíticos, foi reorganizada em modelos dbt individuais e dependentes entre si.

No projeto, scripts auxiliares em Python, denominados `parse_stg.py` e `parse_marts.py`, foram utilizados durante esse processo de migração para decompor os scripts SQL anteriores e transformá-los em aproximadamente **90 modelos dbt**. Os modelos são organizados de acordo com sua responsabilidade dentro da arquitetura.

5.6. Processamento das Evidências e Engenharia de Atributos

A qualidade dos modelos de Machine Learning e dos relatórios de BI depende diretamente da engenharia de atributos realizada no dbt durante a transição da camada Silver para Ouro.

Um aspecto relevante da solução é o tratamento do campo de evidências existente na tabela de tarefas do MUNKA.

5.6.1. Parser de Evidências HTML/Textuais via RegEx no Snowflake

O campo EVIDENCIAS da tabela de tarefas contém blocos de texto formatados em HTML com informações valiosas, porém não estruturadas. Na camada MUNKA_INT (int_tarefa_evidencias_features.sql), aplicamos expressões regulares nativas do Snowflake (REGEXP_COUNT e REGEXP_SUBSTR):

```
-- Exemplo de extração de features textuais na camada Intermediate
SELECT
  tarefa_id,
  REGEXP_COUNT(evidencias, '(?i)<a\\s+[^>]*href') AS qtd_links_evidencia,
  REGEXP_COUNT(evidencias, '(?i)<img\\s+[^>]*src') AS qtd_imagens_evidencia,
  REGEXP_COUNT(evidencias, '(?i)<code>|<pre>') AS qtd_blocos_codigo_evidencia,
  REGEXP_COUNT(evidencias, '(?i)github\\.com|gitlab\\.com|commit') AS qtd_referencias_git,
  LENGTH(COALESCE(evidencias, '')) AS tamanho_texto_evidencia
FROM {{ ref('stg_anexos') }}
```

Figura 6. Comandos de RegEX para obter features.

Nas primeiras versões do processamento, o conteúdo das evidências era representado por características relativamente simples, como tamanho do texto e identificação da presença de determinados elementos.

Durante a evolução do projeto, foi ampliada a extração de atributos por meio de expressões regulares executadas diretamente no Snowflake. O processamento foi incorporado ao modelo: int_tarefa_evidencias_features.sql e passou a produzir atributos derivados do conteúdo das evidências.

Entre as características estão indicadores relacionados à identificação de tecnologias mencionadas nos textos, como Python, SQL e React, bem como ocorrências relacionadas a erros e manutenção de software, incluindo termos como Traceback, Exception e NullPointerException.

Também foram introduzidos atributos relacionados à identificação de referências a Pull Requests e outros elementos encontrados no conteúdo registrado.

Essa estratégia caracteriza um processo de **engenharia de atributos textuais baseada em regras**, realizado por meio de expressões regulares.

Portanto, embora os documentos do projeto utilizem o termo NLP para essa etapa, a implementação atual não utiliza modelos linguísticos, embeddings ou modelos estatísticos de Processamento de Linguagem Natural. O que foi efetivamente implementado é a transformação de conteúdo textual e HTML em variáveis quantitativas e indicadores binários por meio de regras e expressões regulares.

O processamento ocorre diretamente no Snowflake, seguindo uma estratégia de *push-down processing*: as transformações são executadas no Data Warehouse e apenas os atributos já estruturados são entregues à camada de Machine Learning.

5.6.2. Atributos Temporais e Derivados

Duração da Sprint: Cálculo da janela temporal da sprint em dias úteis (DATEDIFF('day', data_início, data_fim)).

Prazo Planejado vs Executado: Proporção entre horas estimadas e prazo contratual.

Componentes de Data: Extração de dia da semana, mês, trimestre e ano para capturar sazonalidade de entregas da equipe.

5.6.3. Atributos Categóricos Encodados e Agregados

Codificação Ordinal da Complexidade: Mapeamento de complexidades (MUITO BAIXA=1, BAIXA=2, MÉDIA=3, ALTA=4, MUITO ALTA=5).

Métricas da UST (Unidade de Serviço Técnico): Fator de conversão e valor unitário da UST aplicável por contrato e perfil profissional.

5.7. Qualidade e Governança dos Dados

A qualidade dos dados foi incorporada ao projeto por meio dos mecanismos de teste do dbt.

Foram definidos arquivos schema.yml para documentar os modelos e associar testes às respectivas colunas.

Entre os testes implementados encontram-se principalmente:

- unique;
- not_null.

Esses testes são utilizados especialmente sobre as chaves das dimensões e fatos para verificar se os identificadores gerados mantêm as propriedades esperadas de unicidade e preenchimento.

A adoção dos testes dbt substitui verificações anteriormente realizadas por scripts SQL isolados e permite que as validações façam parte do próprio fluxo de transformação.

A execução de dbt test foi adicionada ao processo de transformação, de forma que a qualidade das estruturas analíticas possa ser verificada antes de sua utilização pelos componentes de Machine Learning e visualização.

6. Solução de Aprendizagem de Máquina

O modelo desenvolvido nesta primeira versão possui caráter retrospectivo.

O conjunto de atributos inclui informações estruturadas das tarefas e características extraídas de suas evidências de execução. Por essa razão, algumas das features utilizadas somente se tornam disponíveis durante ou após a realização da atividade.

O objetivo do modelo atual não é estimar antecipadamente o esforço de uma tarefa ainda não iniciada, mas produzir uma estimativa de referência baseada no comportamento histórico e nas características observadas durante a execução.

Essa estimativa poderá ser comparada às horas efetivamente registradas, auxiliando na identificação de tarefas com comportamento atípico ou divergente do padrão histórico.

A variável-alvo é “HORAS_EXECUTADAS”, correspondente à quantidade de horas de trabalho registrada para a conclusão da tarefa.

O conjunto de atributos utilizado pelo modelo foi obtido a partir da tabela ML_TAREFA_FEATURES, construída pelo dbt na camada MUNKA_ML. O modelo utiliza 15 features detalhadas na Seção 6.1, incluindo fatores quantitativos e atributos derivados das evidências de execução.

A abordagem adotada utiliza redes neurais do tipo Multilayer Perceptron — MLP.

Para atender ao requisito de comparação entre uma implementação manual e uma implementação baseada em biblioteca, foram desenvolvidas duas abordagens e ambas foram configuradas com Camadas Densas configuráveis, ativação ReLU nas camadas ocultas e Identidade na saída.

Utilizamos os seguintes modelos de treinamento:

- MLP NumPy (implementação manual): rede neural implementada diretamente com NumPy, incluindo forward propagation, backpropagation e atualização dos pesos. A versão utilizada possui duas camadas ocultas e treinamento por SGD, permitindo demonstrar os fundamentos matemáticos do algoritmo e compará-los com uma implementação baseada em biblioteca.
- MLP Scikit-Learn (configuração livre): implementação com MLPRegressor, utilizando o otimizador Adam e espaço de busca de hiperparâmetros mais amplo, permitindo variar número de camadas, unidades ocultas e regularização durante o HPO.
- MLP Scikit-Learn restrito (controle comparativo): implementação com MLPRegressor cujo espaço de busca arquitetural foi limitado para aproximar as condições topológicas avaliadas no modelo NumPy. A comparação permite observar diferenças de desempenho entre as implementações, sem atribuir causalmente o resultado a um único componente, pois permanecem diferenças de otimização, inicialização e detalhes numéricos.

O primeiro consiste em uma MLP implementada manualmente utilizando NumPy, incluindo as operações matemáticas necessárias ao treinamento da rede. O segundo utiliza o MLPRegressor, disponibilizado pela biblioteca Scikit-Learn (SCIKIT-LEARN DEVELOPERS, s.d.).

Dessa forma, o mesmo tipo de modelo pode ser analisado a partir de duas perspectivas: uma implementação construída diretamente a partir das operações matemáticas da rede neural e outra utilizando um framework consolidado de Machine Learning.

Durante a preparação dos dados é utilizado o StandardScaler para normalização das variáveis. Houve cuidado em ajustar o normalizador exclusivamente sobre o conjunto de treinamento, utilizando fit_transform nos dados de treino e somente transform nos conjuntos de validação e teste. Essa separação procura evitar vazamento estatístico de informações durante a padronização.

Para o projeto, utilizamos o Optuna (HPO) como mecanismo de otimização dos **hiperparâmetros**; a avaliação da capacidade de generalização dos modelos foi realizada com **Validação Cruzada em 5 Folds (5-Fold Cross-Validation)**; e o MLflow é o mecanismo de rastreabilidade dos experimentos.

6.1. Preparação dos Dados, Prevenção de Data Leakage e Divisão (Data Splitting)

A etapa de preparação dos dados foi estruturada com o objetivo de garantir a consistência dos atributos utilizados na modelagem, a adequada transformação das variáveis e, principalmente, a prevenção de vazamento de dados (*Data Leakage*) entre os conjuntos utilizados para treinamento, validação e avaliação dos modelos.

6.1.1 Seleção de Atributos e Tratamento de Variáveis

As features utilizadas neste experimento não foram limitadas àquelas disponíveis antes do início da tarefa. O modelo possui finalidade retrospectiva e utiliza informações provenientes das evidências de execução como elementos explicativos do esforço observado.

Por esse motivo, atributos como TEM_COMMIT, FL_TEM_PULL_REQUEST, QTD_BLOCOS_CODIGO, QTD_IMAGENS e TAMANHO_TEXTO são considerados válidos para o cenário analítico atual.

Para a construção da matriz de atributos de entrada (X), foram selecionadas 15 features provenientes da camada Gold MUNKA_ML, previamente tratadas e enriquecidas ao longo do pipeline de dados. As variáveis utilizadas foram: FATOR_AJUSTE, HET_MAX, QTD_IMAGENS, QTD_LINKS, TEM_CODIGO, TEM_SQL, TEM_COMMIT, TEM_ANEXO, FL_ENVOLVE_FRONTEND, FL_ENVOLVE_BACKEND, FL_ENVOLVE_DADOS, FL_IS_BUGFIX,

QTD_BLOCOS_CODIGO, FL_TEM_PULL_REQUEST e TAMANHO_TEXTO. Esses atributos representam características quantitativas e indicadores relacionados à complexidade, ao conteúdo técnico e às evidências associadas às tarefas.

Campos utilizados exclusivamente para identificação, descrição ou contextualização das tarefas não foram incorporados diretamente como variáveis de entrada dos modelos. Entre os atributos descartados encontram-se TAREFA_ID, NOME_TAREFA, NOME_PROJETO, SPRINT_OBJETIVOS, NOME_COMPLEXIDADE, TOTAL_UST e SCORE_QUALIDADE_EVIDENCIA. A exclusão desses campos teve como objetivo evitar a introdução de informações sem representação numérica adequada, identificadores sem capacidade preditiva direta ou atributos que pudessem produzir ruído no processo de aprendizagem.

Os valores ausentes remanescentes foram tratados por meio de imputação com valor zero, utilizando `df.fillna(0)`, garantindo que nenhuma observação apresentasse valores nulos no momento do treinamento. Posteriormente, as variáveis numéricas foram submetidas à padronização por meio do `StandardScaler`, transformando-as para uma escala centrada em média zero ($\mu = 0$) e desvio-padrão unitário ($\sigma = 1$).

6.1.2 Prevenção de Data Leakage na etapa de padronização

Um cuidado específico foi adotado para impedir a ocorrência de *Data Leakage* durante essa padronização. Os parâmetros estatísticos utilizados pelo `StandardScaler`, especialmente média e desvio-padrão, foram calculados exclusivamente a partir dos dados de treinamento. Dessa forma, a operação `fit_transform` foi executada somente sobre o conjunto de treino, enquanto os conjuntos destinados à validação e ao teste receberam apenas a operação `transform`. Esse procedimento impede que informações estatísticas provenientes de observações futuras ou destinadas à avaliação influenciem o processo de aprendizagem dos modelos.

6.1.3 Particionamento Hierárquico dos Dados e Origem das Amostras

Para eliminar qualquer ambiguidade ou sobreposição metodológica, a origem e o destino de cada amostra do dataset de **5.000 registros** extraídos da tabela `MUNKA_ML.ML_TAREFA_FEATURES` no Snowflake são estritamente mapeados no código fonte pelos scripts [src/ml/dataset.py](#), [src/ml/train.py](#), [src/ml/hpo.py](#) e [src/ml/export_evaluation_dataset.py](#).

Para assegurar a reprodutibilidade dos experimentos, foi adotada de forma consistente a semente pseudoaleatória `random_state = 42`, utilizada nas operações de particionamento e validação (`KFold`, `train_test_split` e `np.random.seed(42)`).

Como referência mínima de desempenho foi estabelecido um modelo de Regressão Linear Simples (**Baseline de referência**), treinado sobre os dados padronizados. Esse modelo baseline apresentou $MSE = 345.12$, $MAE = 15.20$ e $R^2 = 0.58$, atuando como o patamar estatístico de comparação obrigatório a ser superado pelas redes neurais MLP.

A divisão primária ocorre via `train_test_split(X, y, test_size=0.2, random_state=42)`:

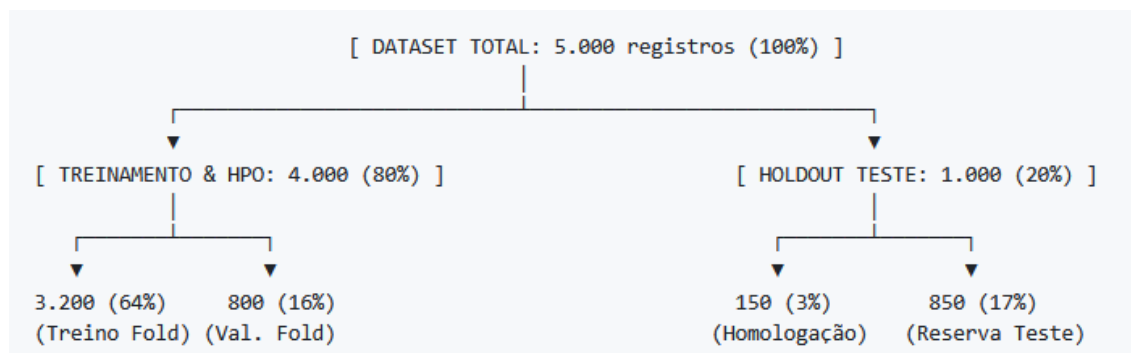


Figura 7. Distribuição dos datasets nas camadas de treinamento.

1. Partição de Treinamento e HPO (80% / 4000 registros):

- Destinada estritamente ao ajuste dos pesos das redes neurais e à busca do Optuna ([src/ml/hpo.py](#)).
- Validação Cruzada (5-Fold Cross-Validation):** Executada exclusivamente sobre os 4.000 registros via `KFold(n_splits=5, shuffle=True, random_state=42)` em [src/ml/train.py](#), subdividindo a partição de treino a cada iteração em **3.200 registros** (64% do dataset total) para treino interno e **800 registros** (16% do dataset total) para validação do MSE.

2. Partição de Teste Holdout Geral (20% / 1000 registros):

- Mantida totalmente intocada e isolada de todas as etapas de treino, HPO e cálculo de parâmetros do StandardScaler.
- Lote Formal de Homologação Auditável (150 registros / 3% do dataset total ou 15% do Holdout):** Amostra real extraída da partição de teste Holdout (`X_test`) com semente reproduzível pelo script [src/ml/export_evaluation_dataset.py](#).
- Reserva de Teste Adicional (850 registros / 17% dataset total ou 85% do Holdout):** Registros remanescentes mantidos no conjunto de teste Holdout como reserva estática.
- Análise Retrospectiva em Lote: o script `src/ml/batch_inference.py` aplica os modelos sobre tarefas já executadas no Snowflake (`WHERE HORAS_EXECUTADAS IS NOT NULL`), gera estimativas retrospectivas e calcula os desvios em relação às horas efetivamente registradas. Essa

etapa possui finalidade analítica e não representa previsão antecipada de tarefas abertas.

6.2. Otimização de Hiperparâmetros

O projeto incorporou o Optuna para experimentação com Hyperparameter Optimization — HPO. Optuna é uma biblioteca de Python voltada para otimização automática de hiperparâmetros — o chamado HPO (Hyperparameter Optimization). Em vez de testar manualmente combinações, o Optuna executa vários experimentos, compara os resultados e vai escolhendo combinações cada vez mais promissoras. (OPTUNA, s.d.)

A estratégia de otimização considera parâmetros relacionados à arquitetura e ao treinamento das redes MLP, incluindo:

- taxa de aprendizado;
- quantidade de camadas;
- quantidade de unidades nas camadas ocultas;
- regularização, quando aplicável.

A arquitetura de otimização é a seguinte:

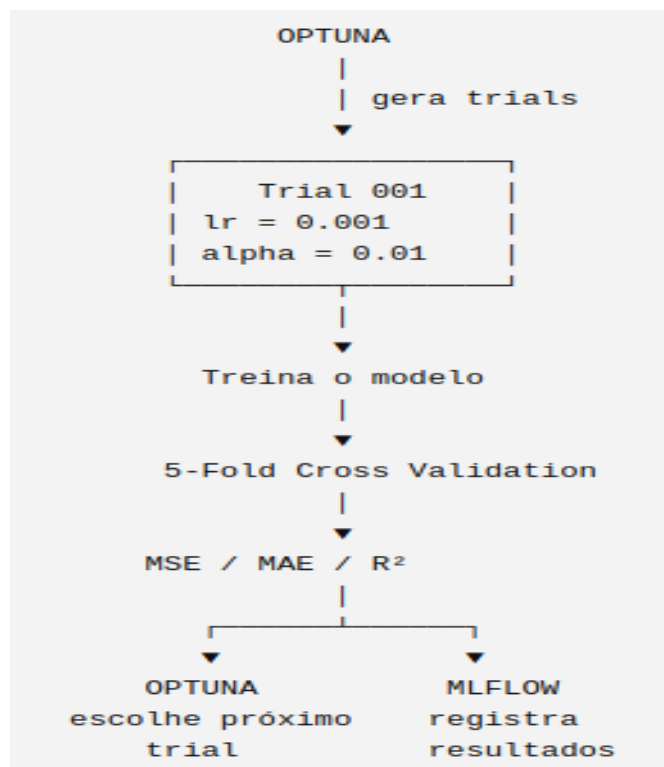


Figura 8. Arquitetura de Otimização com o uso do Optuna.

Os artefatos de avaliação registram os parâmetros, as métricas e a configuração correspondente ao modelo selecionado, permitindo rastrear a decisão de seleção.

Dessa forma, a seleção do modelo não foi baseada simplesmente na existência de uma configuração otimizada. O projeto preservou o modelo de melhor desempenho observado na avaliação registrada.

A otimização de hiperparâmetros foi executada automaticamente com os seguintes parâmetros no HPO:

- 1) **Espaço de Busca (Sklearn):** learning_rate de 10^{-4} a 10^{-1} ; 1 a 3 camadas; de 8, 16, 32, 64 ou 128 neurônios por camada e a regularização (alfa) 10^{-5} a 10^{-1} .

```
3 def objective_sklearn(trial, X_train, X_val, y_train, y_val):
4     lr = trial.suggest_float("learning_rate", 1e-4, 1e-1, log=True)
5     n_layers = trial.suggest_int("n_layers", 1, 3)
6     layers = []
7     for i in range(n_layers):
8         n_units = trial.suggest_categorical(f"n_units_l{i}", [8, 16, 32, 64, 128])
9         layers.append(n_units)
10    hidden_sizes = tuple(layers)
11    alpha = trial.suggest_float("alpha", 1e-5, 1e-1, log=True)
```

Figura 9. Parametrização do HPO para o sklearn no hpo.py .

- 2) **Espaço de Busca (NumPy):** learning_rate de 10^{-4} a 10^{-2} , de uma a duas camadas e de 8 a 64 neurônios por camada.

```
def objective_numpy(trial, X_train, X_val, y_train, y_val):
    lr = trial.suggest_float("learning_rate", 1e-4, 1e-1, log=True)
    size_l1 = trial.suggest_categorical("n_units_l1", [16, 32, 64])
    size_l2 = trial.suggest_categorical("n_units_l2", [8, 16, 32])
    hidden_sizes = (size_l1, size_l2)
```

Figura 10. Parametrização do HPO para o NumPy no hpo.py.

- 3) **Espaço de Busca (Scikit-Learn MLP Restrito):** learning_rate de 10^{-4} a 10^{-2} , de uma a duas camadas e de 8 a 64 neurônios por camada, equivalente ao do Numpy.

```
def objective_sklearn_restricted(trial, X_train, X_val, y_train, y_val):
    lr = trial.suggest_float("learning_rate", 1e-4, 1e-1, log=True)
    size_l1 = trial.suggest_categorical("n_units_l1", [16, 32, 64])
    size_l2 = trial.suggest_categorical("n_units_l2", [8, 16, 32])
    hidden_sizes = (size_l1, size_l2)
    # alpha is fixed to default 0.0001 to match numpy constraint
```

Figura 11. Parametrização do HPO para o Scikit-Learn MLP restrito no hpo.py.

A avaliação da capacidade de generalização dos modelos foi realizada com **Validação Cruzada em 5 Folds (5-Fold Cross-Validation)**.

6.3. Rastreamento e Registros no MLflow.

O projeto utiliza o MLflow como componente de rastreamento dos experimentos de Machine Learning. O MLflow é uma plataforma para gerenciar o ciclo de vida dos experimentos de Machine Learning. Ele serve principalmente para registrar, comparar, reproduzir e organizar treinamentos de modelos. Os experimentos são registrados em um banco SQLite local, por meio da configuração: `sqlite:///mlflow.db` (MLFLOW PROJECT, s.d.)

O MLflow é utilizado para armazenar informações produzidas durante os treinamentos, incluindo:

- parâmetros dos modelos;
- hiperparâmetros;
- métricas de desempenho;
- valores de loss durante o treinamento;
- artefatos produzidos pelos experimentos;
- gráficos associados às avaliações.

Essa abordagem permite manter um histórico das execuções e comparar diferentes configurações de treinamento.

Foram documentados experimentos relacionados tanto ao modelo baseline quanto às execuções de otimização de hiperparâmetros.

No treinamento comparativo atual, o MLflow registra a execução retrospectiva no experimento ``MUNKA_MLP_Retrospective``, juntamente com parâmetros, métricas de K-Fold e holdout e o contrato metodológico das features utilizadas.

* Contrato das features utilizadas (``feature_contract.json``) e parâmetros metodológicos do experimento.

* Gráfico comparativo de resíduos e curva de aprendizado (``residuals_comparison.png`` e ``loss_validation_curve.png``).

* Gráficos de importância por permutação (``feat_imp_numpy.png`` e ``feat_imp_sklearn.png``), além das métricas registradas no MLflow. Os artefatos do modelo campeão e do escalador são produzidos pelo fluxo de retreinamento do Passo 5.

6.4. Métricas de Avaliação e Resultados Comparativos

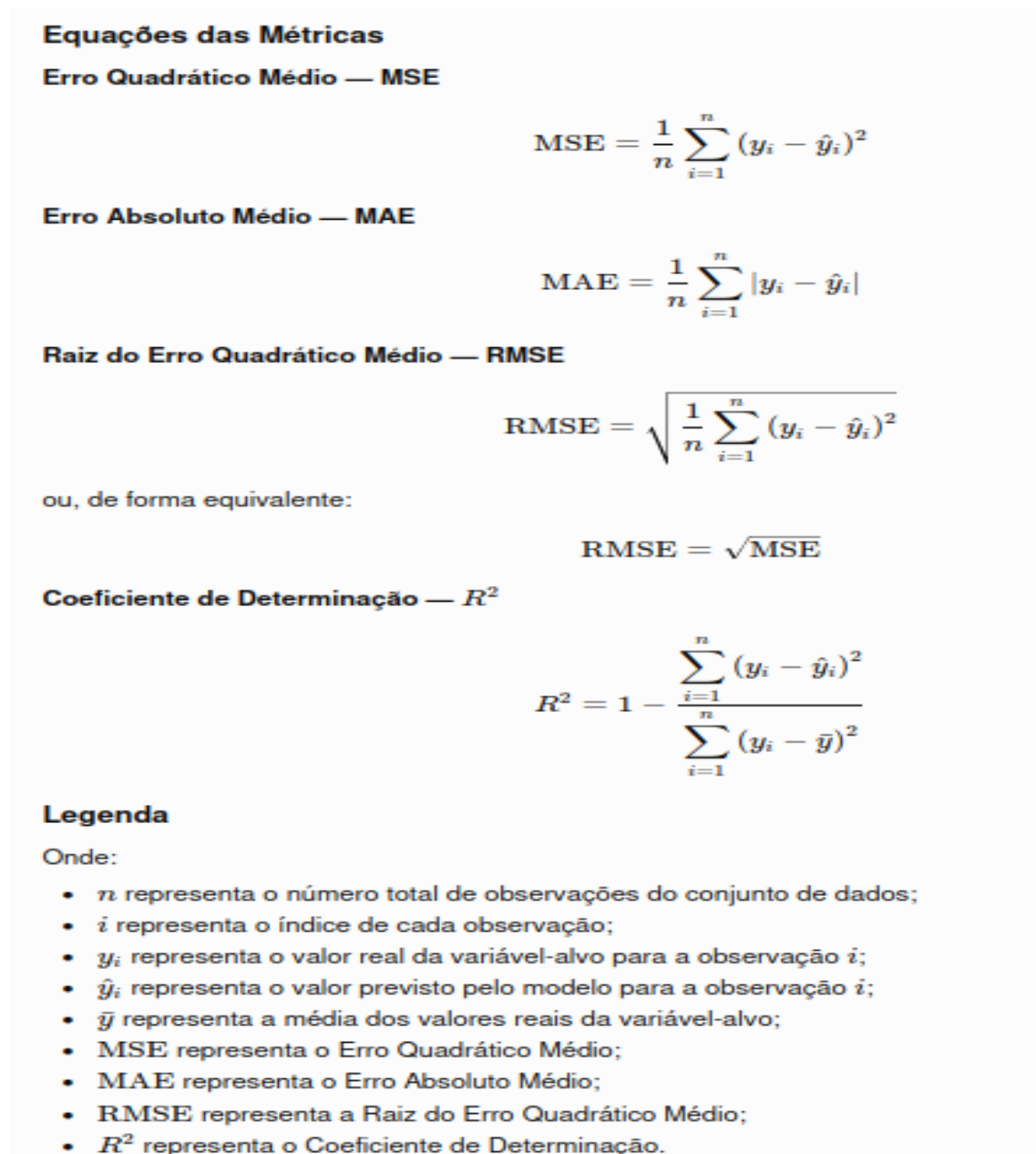


Figura 12. Métricas para avaliação.

No contexto do projeto, y_i corresponde à quantidade real de horas executadas em uma tarefa e $\hat{y}_i$ corresponde à quantidade de horas estimada pelo modelo de Aprendizagem de Máquina. Dessa forma, MAE e RMSE são expressos em horas, o MSE em horas² e o R^2 é uma medida adimensional que indica a proporção da variabilidade da variável-alvo explicada pelo modelo.

6.5. Análise Comparativa Detalhada: Scikit-Learn (Livre e Restrito) vs. NumPy

A matriz comparativa a seguir apresenta os resultados das diferentes estratégias e restrições topológicas avaliadas. Na comparação restrita, o Scikit-Learn obteve MSE = 5,49, enquanto a implementação NumPy obteve MSE = 6,90. O resultado indica melhor desempenho da implementação Scikit-Learn nas condições avaliadas, mas não permite atribuir essa diferença exclusivamente ao otimizador, à linguagem de implementação ou a um único componente do treinamento.

Algoritmo / Implementação	Configuração Inicial (Sem HPO)	Configuração Otimizada (Com HPO)	Status Final
Scikit-Learn MLP (Modelo Escolhido)	MLP Base Scikit-Learn • Topologia: (32, 16) • MSE: 4.55 [*] • R^2 : 0.75	MLP HPO Scikit-Learn (Campeão) • Topologia Livre: (64, 128) • MSE: 5.46 • Teste Final: MAE: 0.67 • Teste Final: R^2 : 0.75	Modelo final selecionado
Scikit-Learn MLP Restrito (sklearn_restricted)	(N/A — Usado apenas no HPO)	MLP HPO Sklearn Restrito • Topologia Fixa (2 cam.): (64, 32) • MSE: 5.49	Grupo de Controle Científico
NumPy MLP (Implementação Própria)	MLP Base NumPy • Topologia: (32, 16) • MSE: 6.04 [*] • R^2 : 0.67	MLP HPO NumPy (Vice-Campeão) • Topologia Fixa (2 cam.): (16, 32) • MSE: 6.90	Vice-Campeão Matemático
Baseline de referência	Regressão Linear • MSE: 345.12 • R^2 : 0.58	(N/A — Modelo Linear Fixo)	Benchmark Estatístico

*Nota: Os valores Base referem-se a testes preliminares de código antes do HPO final.

Tabela 3. Comparativo entre os modelos de ML.

Nome do Modelo	Estratégia / Topologia	MSE Validação	MAE	R^2 Score	Papel / Status
MLP HPO Scikit-Learn	2 camadas (64, 128), $lr=0.0033$, $\alpha=0.000025$	5.46	0.67	0.75	🏆 Modelo final selecionado
Scikit-Learn MLP Restrito (<code>sklearn_restricted</code>)	2 camadas (64, 32), $lr=0.0027$, $\alpha=0.0001$	5.49	--	--	🔧 Controle (Apples-to-Apples)
MLP HPO NumPy	2 camadas (16, 32), $lr=0.0375$	6.90	--	--	🥈 Vice-Campeão
MLP Base Scikit-Learn	2 camadas (32, 16), $lr=0.01$	4.55*	--	0.75	Baseline MLP Sklearn
MLP Base NumPy	2 camadas (32, 16), $lr=0.01$	6.04*	--	0.67	Baseline MLP NumPy
Baseline de referência	Regressão Linear Simples	345.12	15.20	0.58	Baseline Estatístico

*Nota: Os valores de MSE dos modelos Base correspondem a rodadas com escalonamento direto e amostras reduzidas de validação inicial.

Tabela 4. Comparativo Completo dos Experimentos (Registrado no MLflow).

Estas são as principais conclusões:

- Comparação sob restrições topológicas semelhantes: o Scikit-Learn restrito apresentou MSE = 5,49, inferior ao MSE = 6,90 da implementação NumPy. A diferença observada caracteriza o desempenho das implementações neste experimento e deve ser interpretada considerando também diferenças de otimizador, inicialização, critérios de parada, batching e implementação numérica.
- Configuração livre do Scikit-Learn: no espaço de busca mais amplo, o HPO selecionou topologia (64, 128) e regularização L2, alcançando MSE = 5,46 na validação utilizada para seleção. Esse resultado foi o melhor entre as configurações HPO comparadas nesse protocolo.
- Implementação manual em NumPy: o modelo NumPy apresentou capacidade de aprendizagem e resultados compatíveis com uma implementação funcional de MLP, cumprindo o objetivo acadêmico de implementar e avaliar manualmente as etapas de forward propagation e backpropagation. O resultado, contudo, não constitui prova isolada de equivalência matemática ou de desempenho entre implementações distintas.

O modelo final foi selecionado comparando os modelos sob o mesmo conjunto de treinamento e o mesmo protocolo de validação cruzada. Somente após a seleção o modelo foi avaliado no holdout.

Os valores de MSE apresentados nas tabelas de comparação do HPO correspondem ao protocolo de validação empregado durante a seleção dos modelos. As métricas do lote formal de 150 registros apresentadas a seguir correspondem ao holdout de avaliação e, por representarem contextos distintos, não devem ser comparadas diretamente como se fossem a mesma etapa experimental.

Foi criado um conjunto formal de avaliação do modelo, armazenado na estrutura `src/ml/evaluation`. Os artefatos documentados incluem:

- `X_test.csv`;
- `y_test.csv`;
- `predictions.csv`;
- `metrics.json`;
- `analise_qualitativa_erros.csv`.

O arquivo final de métricas registra uma amostra de teste com 150 observações.

Para essa avaliação foram registrados os seguintes resultados: $MAE = 2,0449$ h, $MSE = 6,7091$ h², $RMSE = 2,5902$ h e $R^2 = 0,9114$.

O coeficiente de determinação registrado indica que, no conjunto de avaliação utilizado, o modelo conseguiu explicar aproximadamente 91,14% da variabilidade observada na variável-alvo.

Entretanto, esse resultado deve ser interpretado dentro das características e limitações do conjunto de dados utilizado, não representando garantia de desempenho equivalente em outros registros históricos ou em dados com distribuições diferentes daquelas observadas durante o desenvolvimento.

A estrutura de avaliação também inclui o registro das estimativas individuais e dos respectivos erros, permitindo a análise qualitativa dos casos com maior e menor diferença entre o valor previsto e o valor observado.

6.6. Diagnóstico dos Modelos por Curvas de Aprendizado e Resíduos

Além das métricas agregadas, foram analisadas as curvas de aprendizado e os gráficos de resíduos gerados para as configurações de melhores hiperparâmetros dos modelos NumPy MLP, Scikit-Learn MLP e Scikit-Learn MLP restrito. Esses artefatos complementam a avaliação numérica ao permitir observar o comportamento do processo de treinamento e a distribuição dos erros em função das horas reais.

6.6.1. Curvas de aprendizado

Na implementação NumPy, as curvas de treino e validação apresentam redução acentuada do erro nas primeiras épocas. A curva de validação permanece acima da curva de treinamento durante o processo. Após a fase inicial de convergência, o erro de treinamento continua diminuindo gradualmente, enquanto a validação passa a apresentar oscilações em torno de um patamar superior. No gráfico apresentado não se observa uma tendência sustentada de crescimento do erro de validação; o comportamento observado é de estabilização com maior variabilidade em relação ao conjunto de treinamento.

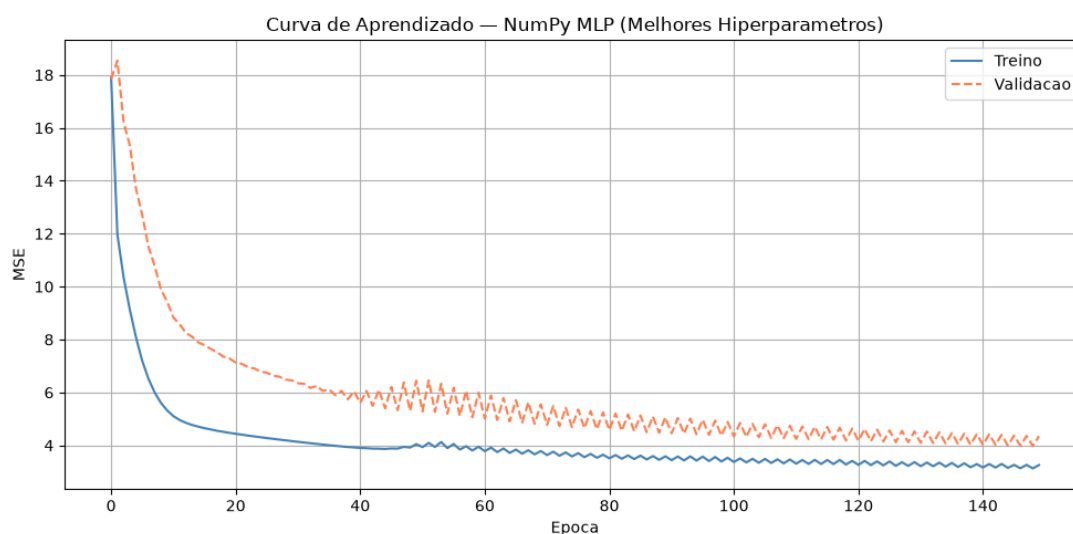


Figura 13. Curva de aprendizado do NumPy MLP com os melhores hiperparâmetros.

Para o Scikit-Learn MLP na configuração livre, a curva de treinamento apresenta queda contínua e suave ao longo das épocas, sem as oscilações observadas na curva de validação do NumPy. O artefato disponibilizado contém apenas a trajetória de treinamento; por esse motivo, ele permite avaliar a convergência do processo, mas não permite, isoladamente, medir a distância entre treino e validação ou concluir sobre sobreajuste a partir dessa figura.

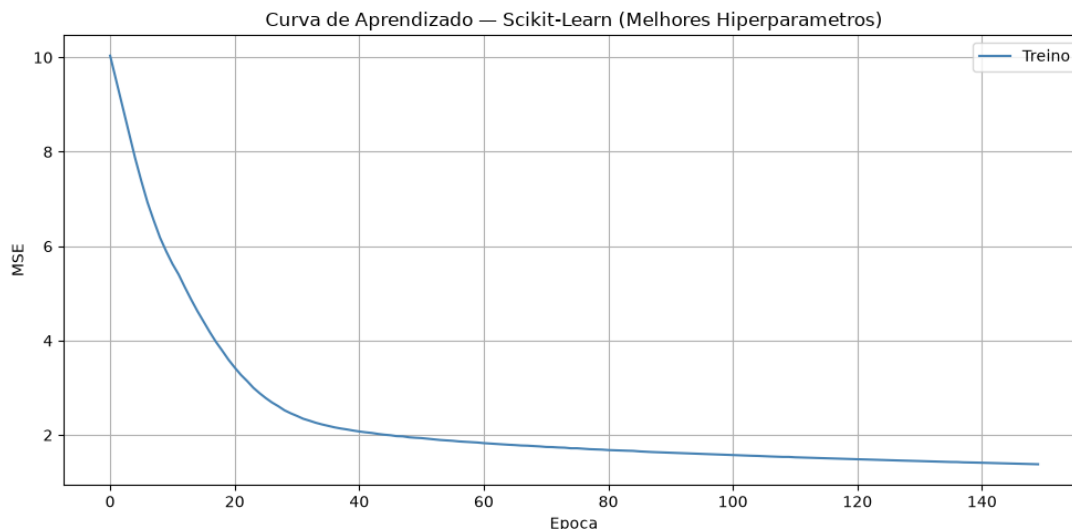


Figura 14. Curva de aprendizado do Scikit-Learn MLP com os melhores hiperparâmetros.

O Scikit-Learn MLP restrito também apresenta redução contínua e suave da função de perda, alcançando estabilização em um número menor de épocas no artefato apresentado. Assim como no gráfico do Scikit-Learn livre, não há curva de validação exibida, de modo que a comparação visual deve ser utilizada para caracterizar a trajetória de otimização, e não como prova isolada de maior capacidade de generalização.

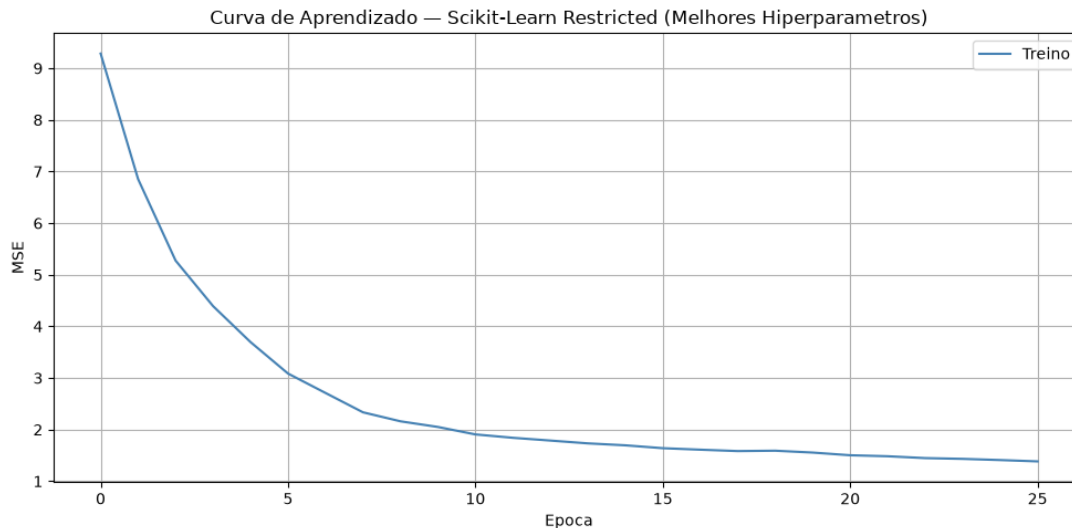


Figura 15. Curva de aprendizado do Scikit-Learn MLP restrito com os melhores hiperparâmetros.

Em conjunto, as curvas mostram que as três implementações efetivamente reduzem o erro durante o treinamento. Visualmente, as trajetórias das implementações Scikit-Learn são mais suaves, enquanto a implementação NumPy apresenta maior oscilação na validação. Como os artefatos possuem números de épocas e configurações distintas, essas curvas não permitem atribuir as diferenças observadas a um único fator, como linguagem, otimizador ou biblioteca; elas documentam o comportamento obtido nas configurações efetivamente avaliadas.

6.6.2. Análise dos resíduos

Os resíduos foram calculados como diferença entre o valor estimado e o valor real (resíduo = estimado - real). Assim, resíduos positivos correspondem a superestimação e resíduos negativos correspondem a subestimação. Em uma situação sem padrão sistemático, espera-se que os pontos se distribuam ao redor da linha zero sem estrutura evidente.

No NumPy MLP, observa-se grande concentração de resíduos próximos de zero nas tarefas de menor duração. Também aparece um ponto de superestimação muito elevado e, à medida que aumentam as horas reais, tornam-se mais frequentes resíduos negativos. O padrão indica que, nos registros de maior duração representados no gráfico, o modelo tende com maior frequência a estimar valores inferiores aos efetivamente registrados.

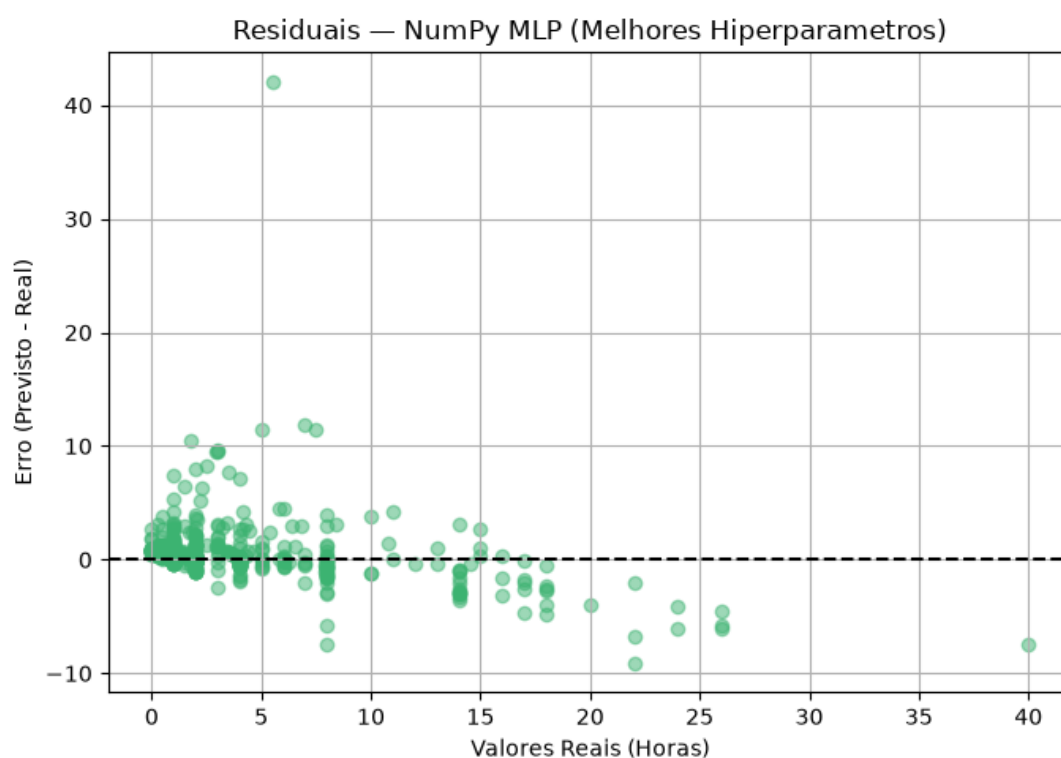


Figura 16. Resíduos do NumPy MLP com os melhores hiperparâmetros.

O Scikit-Learn MLP livre apresenta padrão semelhante: há concentração de observações próximas à linha zero para valores reais menores, presença de um ponto de superestimação elevado e maior ocorrência de resíduos negativos nas tarefas de maior duração. Portanto, o comportamento de subestimação nas faixas mais altas de horas não é exclusivo da implementação NumPy.

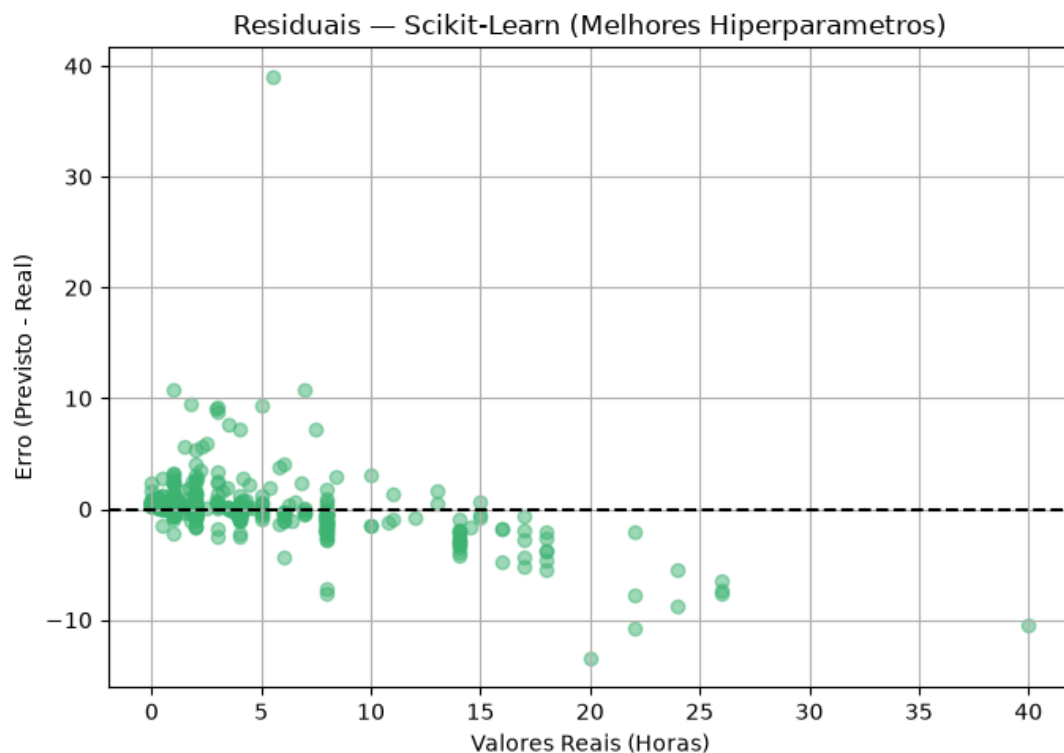


Figura 17. Resíduos do Scikit-Learn MLP com os melhores hiperparâmetros.

O Scikit-Learn MLP restrito repete o mesmo comportamento geral. A maior parte das observações de menor duração permanece próxima da linha de erro zero, enquanto os registros de maior duração apresentam predominantemente resíduos negativos. O gráfico também contém um ponto de superestimação extrema, evidenciando que a restrição arquitetural não eliminou os casos de erro elevado.

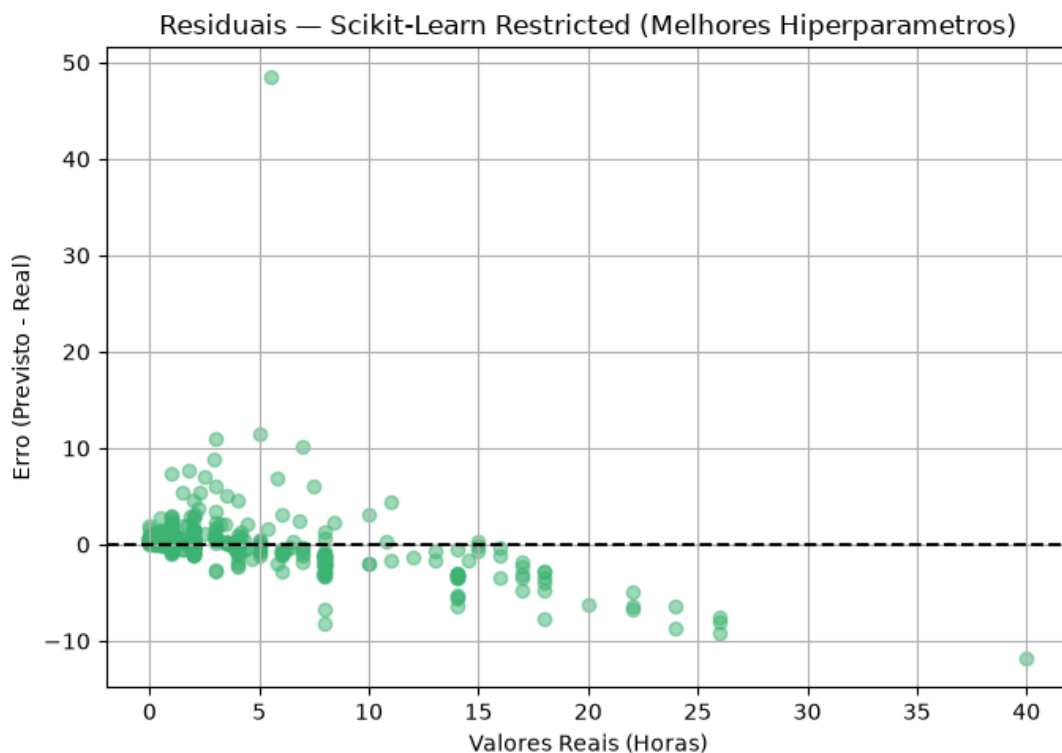


Figura 18. Resíduos do Scikit-Learn MLP restrito com os melhores hiperparâmetros.

6.6.3. Síntese comparativa dos diagnósticos

A comparação dos resíduos evidencia um padrão comum às três implementações: boa concentração de erros próximos de zero nas tarefas de menor duração, presença de poucos casos de superestimação elevada e aumento da frequência de subestimações nas tarefas com maior número de horas reais. Como esse padrão aparece nas três configurações, os gráficos não sustentam a atribuição dos maiores erros exclusivamente a uma implementação específica.

Esse comportamento deve ser interpretado juntamente com o perfilamento já documentado do conjunto de dados, que registrou 142 observações classificadas como outliers pelo critério de $1,5 \times \text{IQR}$ e 18 ocorrências superiores a 100 horas. Os gráficos de resíduos não determinam a causa dos desvios, mas mostram que a qualidade das estimativas não é uniforme em todas as faixas de esforço e justificam a análise específica dos registros de maior duração e dos casos extremos.

6.7. Análise da Importância das Features por Permutação

Para complementar a análise do comportamento dos modelos, foi utilizada Permutation Feature Importance sobre as 15 features empregadas no treinamento. No código do

projeto, a importância é calculada por permutação repetida de cada atributo, utilizando como critério de avaliação o erro quadrático médio por meio do scoring `neg_mean_squared_error`. Nessa abordagem, uma feature recebe maior importância quando a sua permutação degrada mais o desempenho do modelo.

Para interpretação da feature dominante, `HET_MAX` corresponde ao limite máximo de horas definido na regra associada à atividade, proveniente do campo `regra.het_max`. No pipeline, esse atributo é preservado em `stg_regra`, incorporado à `dim_regra` e posteriormente incluído em `ml_tarefa_features`. Trata-se, portanto, de um atributo de regra de negócio expresso em horas, e não de uma característica extraída das evidências textuais.

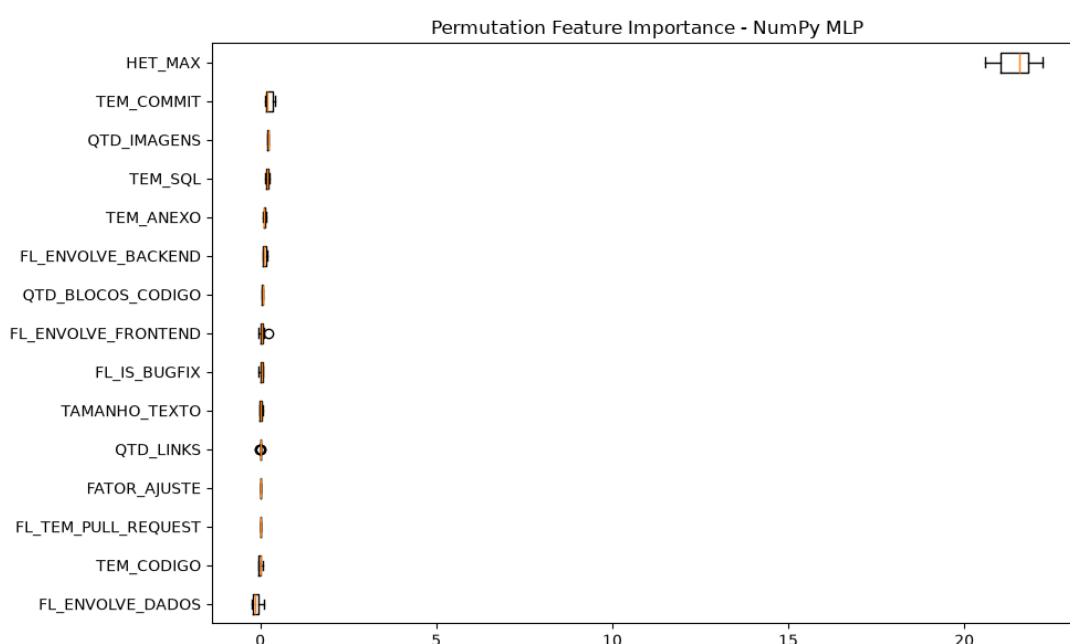


Figura 19. Importância das features por permutação no NumPy MLP.

No NumPy MLP, `HET_MAX` apresenta importância muito superior às demais variáveis. `TEM_COMMIT` aparece como a segunda feature com contribuição positiva mais evidente, porém em escala muito inferior à de `HET_MAX`. As demais variáveis permanecem concentradas próximas de zero, com algumas importâncias ligeiramente negativas.

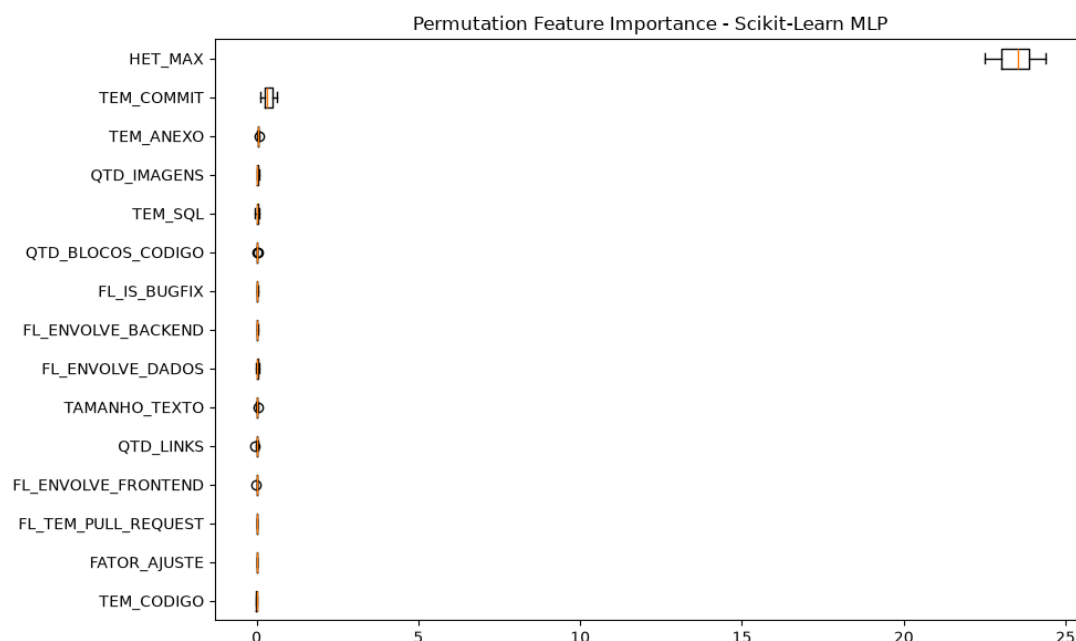


Figura 20. Importância das features por permutação no Scikit-Learn MLP.

O Scikit-Learn MLP reproduz o mesmo resultado principal: HET_MAX é, por ampla margem, a feature de maior importância por permutação, e TEM_COMMIT ocupa a segunda posição com valor muito menor. O grupo formado pelas demais features permanece próximo de zero, embora a ordenação interna dessas variáveis apresente pequenas diferenças em relação ao NumPy.

6.7.1. Comparação entre as implementações

A concordância entre as duas implementações quanto à predominância de HET_MAX é o resultado mais consistente da análise de importância. Também há concordância quanto à contribuição secundária de TEM_COMMIT. Para as demais features, as importâncias reduzidas indicam que, no conjunto e nos modelos avaliados, a permutação individual desses atributos altera pouco o desempenho medido pelo MSE.

Valores de importância próximos de zero ou ligeiramente negativos não devem ser interpretados como prova de irrelevância causal da variável. Eles indicam apenas que, neste experimento e na presença das demais features, embaralhar isoladamente aquele atributo não produziu aumento relevante do erro, ou pôde produzir pequena melhora decorrente da variabilidade do procedimento. Por esse motivo, a remoção de features não deve ser feita somente a partir desses gráficos, sem novo treinamento e validação.

No escopo retrospectivo adotado nesta versão, atributos derivados das evidências de execução, como TEM_COMMIT, permanecem metodologicamente compatíveis com a finalidade do modelo. Para a evolução futura de um modelo antecipado, a seleção de features deverá ser refeita considerando apenas informações disponíveis antes do início da execução.

6.8. Aplicação Retrospectiva em Lote

Após o treinamento, o Passo 6 foi ajustado para executar uma análise retrospectiva sobre tarefas já concluídas no Snowflake. A execução analisada utilizou 100 registros com HORAS_EXECUTADAS preenchida e o conjunto `retrospective\_features\_v1`. O arquivo `analise_retrospectiva_metrics.json` registra explicitamente o escopo `retrospective\_analysis`, o momento `post\_execution` e o uso de evidências de execução.

6.8.1. Resultados agregados da execução

Na execução retrospectiva, o Scikit-Learn apresentou menor erro agregado que a implementação NumPy. Os resultados são apresentados na tabela a seguir.

Métrica	Scikit-Learn	NumPy
MAE (h)	0,5700	0,7365
MSE (h ²)	0,9859	1,3790
RMSE (h)	0,9929	1,1743
R ²	0,9042	0,8660

Tabela 5. Resultados agregados da aplicação retrospectiva em lote.

Em 69 das 100 tarefas analisadas, a estimativa do Scikit-Learn ficou mais próxima das horas registradas; em 31 casos, a implementação NumPy ficou mais próxima. Esses quantitativos foram obtidos diretamente do arquivo `analise\_retrospectiva.csv` da execução oficial de 14/08/2026. Como esses 100 registros foram obtidos diretamente da tabela analítica por filtro de tarefas concluídas e não foram definidos como um novo holdout independente do treinamento, essas métricas possuem finalidade descritiva e demonstrativa da aplicação retrospectiva e não substituem as métricas formais do conjunto de avaliação isolado.

6.8.2. Análise qualitativa de acertos e desvios

A análise por tarefa permite observar casos com elevada aderência entre a estimativa e o valor registrado, bem como situações com desvios relevantes que merecem investigação. A tabela a seguir apresenta exemplos reais da execução sobre o Snowflake.

Categoria	Tarefa	Horas registradas	Estimativa Sklearn	Erro absoluto
Maior aderência	155605	1,50	1,46	0,04
Maior aderência	2818	1,00	0,94	0,06
Maior aderência	82892	1,00	1,07	0,07
Maior aderência	40042	1,00	1,09	0,09
Maior desvio	35647	11,00	15,43	4,43
Maior desvio	18075	4,00	8,39	4,39
Maior desvio	101566	17,00	13,75	3,25
Maior desvio	160116	4,00	1,22	2,78
Maior desvio	17091	1,00	3,54	2,54

Tabela 6. Exemplos de maior aderência e maiores desvios na aplicação retrospectiva.

Nos casos de maior aderência, os erros absolutos ficaram entre 0,04 h e 0,09 h. Entre os maiores desvios observados, destacaram-se as tarefas 35647 e 18075, com erros absolutos de 4,43 h e 4,39 h, respectivamente. Esses casos não devem ser interpretados automaticamente como falhas de execução ou medição: constituem sinais para análise posterior das características da tarefa, da qualidade dos apontamentos e de fatores não representados pelas features atuais.

Foram identificadas 2 tarefas com HORAS_EXECUTADAS igual a zero. Nesses registros, o erro percentual não foi calculado, pois a métrica envolveria divisão por zero; os casos foram mantidos nas métricas absolutas. Possíveis causas para os maiores desvios incluem baixa representatividade de determinados tipos de manutenção, ruído nos apontamentos de horas e ausência de atributos capazes de representar toda a complexidade técnica ou organizacional da atividade.

A aplicação retrospectiva deve ser utilizada como apoio à análise e priorização de casos para revisão, e não como mecanismo automático de validação contratual. O valor estimado pelo modelo não substitui as regras formais de medição nem constitui evidência isolada de erro, impropriedade ou necessidade de glosa.

7. Infraestrutura AWS

O Amazon S3 é utilizado como componente efetivo da arquitetura para armazenamento dos arquivos de dados utilizados no processo de ingestão. (AMAZON WEB SERVICES, s.d.)

Além dessa utilização, foi desenvolvido um template de **Infraestrutura como Código — IaC** utilizando AWS CloudFormation.

O arquivo cloudformation.yaml define recursos relacionados a:

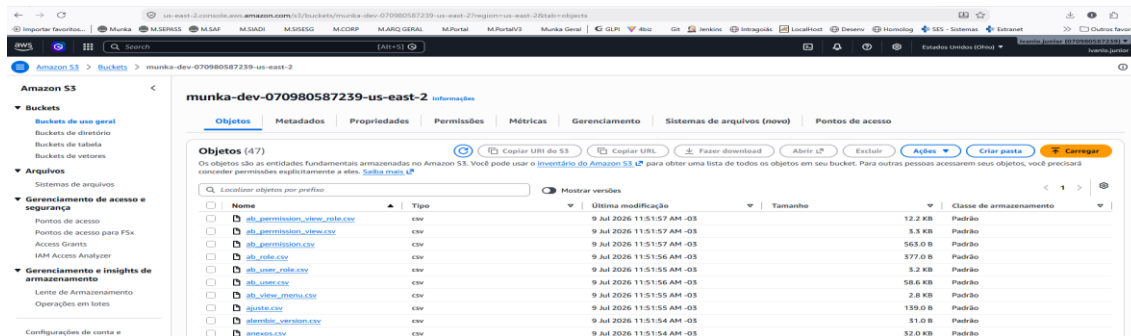
- Amazon S3;
- IAM;

Para o bucket S3, o template estabelece criptografia do lado do servidor com AES-256, bloqueio de acesso público, versionamento e identificação por tags.

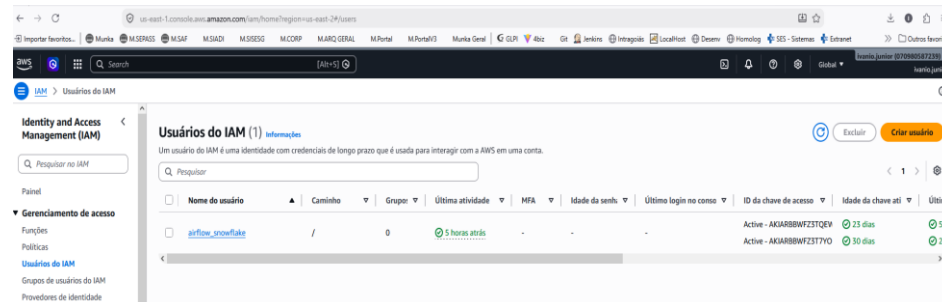
O template também define uma IAM Role e políticas relacionadas ao acesso aos objetos armazenados no S3, além de um CloudWatch Log Group com retenção configurada para 30 dias.

A existência do template permite descrever a infraestrutura de forma declarativa e reproduzível. Os arquivos disponibilizados, entretanto, não constituem por si só evidência de que todos os recursos definidos no CloudFormation tenham sido efetivamente implantados na AWS. Essa comprovação depende das evidências de execução da stack.

7.1. Componentes Utilizados



- **Amazon Simple Storage Service (S3):** Bucket dedicado ([munka-dev-070980587239-us-east-2](#)) configurado na região us-east-2.



- **AWS Identity and Access Management (IAM):** Usuário IAM com política de leitura em tempo de execução (*Least Privilege Principle*).

7.2. Mecanismo de Ingestão de Alta Performance (COPY INTO)

Em vez de trafegar arquivos pela memória do Airflow, o pipeline utiliza a integração nativa entre AWS S3 e Snowflake via comandos SQL auto-gerenciados:

```
-- Ingestão direta de alta velocidade do S3 no Snowflake
COPY INTO DRAGON_DB.MUNKA_RAW.AB_TASK
FROM 's3://munka-dev-070980587239-us-east-2/ab_task.csv'
CREDENTIALS = (
  AWS_KEY_ID = '{{ conn.aws_default.login }}'
  AWS_SECRET_KEY = '{{ conn.aws_default.password }}'
)
FILE_FORMAT = (TYPE = 'CSV' FIELD_OPTIONALLY_ENCLOSED_BY = '"' SKIP_HEADER = 1);
```

Figura 21. Ingestão com o uso do COPY INTO

8. Arquitetura Equivalente Totalmente em AWS

Além da arquitetura desenvolvida com Snowflake, dbt, Airflow e serviços AWS, a documentação do projeto apresenta uma proposta de arquitetura equivalente utilizando serviços nativos da AWS.

Essa arquitetura **não corresponde à implementação atualmente executada**, sendo apresentada como alternativa arquitetural para uma eventual migração integral para a nuvem AWS.

Na proposta, os principais componentes seriam:

- Amazon S3 para armazenamento;
- AWS Glue e Glue Data Catalog para processamento e catálogo de dados;
- Amazon Redshift como Data Warehouse;
- Amazon SageMaker para treinamento e execução dos modelos;
- Amazon QuickSight para visualização;
- Amazon MWAA para execução gerenciada do Apache Airflow;
- Amazon CloudWatch para monitoramento;
- AWS IAM para controle de acesso.

Também existe uma proposta de evolução utilizando **Amazon ECS com AWS Fargate** para desacoplar os processos computacionalmente mais pesados de Machine Learning do container responsável pela execução do Airflow.

Nessa proposta, o Airflow permaneceria responsável apenas pela orquestração, enquanto treinamentos, otimizações e inferências seriam executados em containers independentes no ECS Fargate, com imagens armazenadas no Amazon ECR.

Essa arquitetura deve ser entendida como uma possibilidade de evolução, não como parte já implantada da solução atual.

O diagrama equivalente também incorpora serviços de governança e segurança apresentados na Figura 22, incluindo AWS KMS para gerenciamento de chaves, AWS Lake Formation para governança do Data Lake e AWS CloudTrail para auditoria das ações, complementando IAM e CloudWatch.

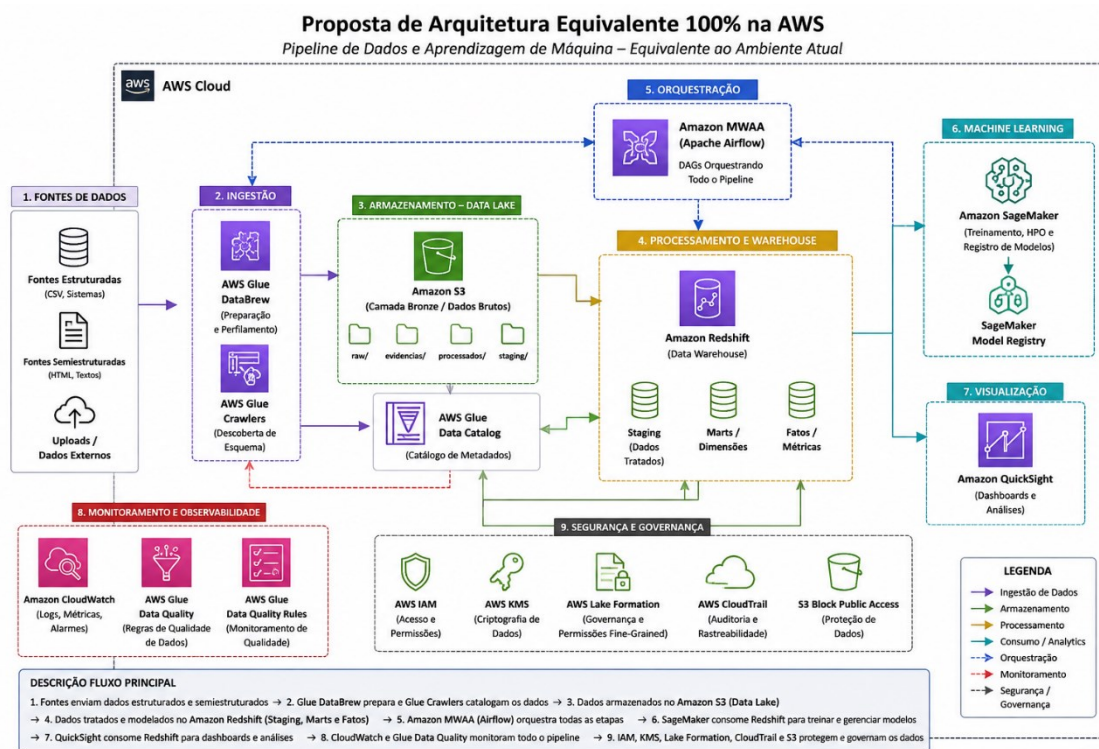


Figura 22. Proposta de Arquitetura Equivalente 100% na AWS

8.1. Estimativa de Custos AWS & Snowflake

Abaixo está a estimativa de custos operacionais mensais projetada para o ambiente de desenvolvimento/homologação do projeto:

Serviço / Componente	Uso Estimado / Parâmetros	Custo Estimado Mensal (USD)
Amazon S3	10 GB (Raw, Processed, ML)	\$0.23
AWS CloudWatch	Logs de auditoria (< 2 GB)	\$1.00
AWS IAM / CloudFormation	Recursos de governança	\$0.00 (Gratuito)
Snowflake Data Cloud	Standard Warehouse X-Small (~2h/dia)	\$15.00
Apache Airflow & Metabase	Containers Docker Locais	\$0.00
TOTAL ESTIMADO		~\$16.23 / mês

Tabela 7. Estimativa de Custos.

9. Visualização com Metabase

O Metabase foi incorporado à arquitetura como ferramenta de Business Intelligence responsável pelo consumo das estruturas analíticas existentes no Snowflake. Ele consome diretamente os dados já modelados no Snowflake (camadas MUNKA_GOLD e MUNKA_ML), permitindo montar dashboards e relatórios analíticos sem escrever código. (METABASE, s.d.)

As fontes de dados utilizadas pelos dashboards são os schemas:

- MUNKA_GOLD;
- MUNKA_ML.

A autenticação dos componentes Airflow, dbt e Metabase junto ao Snowflake utiliza autenticação por par de chaves RSA. O acesso aos recursos AWS utiliza mecanismos próprios do IAM e é independente da autenticação RSA do Snowflake. No Metabase, a chave privada é disponibilizada ao contêiner por caminho configurado, sem inclusão do conteúdo secreto no relatório.

Essa integração disponibiliza tabelas dimensionais, fatos e resultados de Machine Learning para indicadores e recortes analíticos. O Passo 6 gera `analise\_retrospectiva.csv` e o Passo 7 materializa esses dados no Snowflake para consumo pelo Metabase. No dashboard GOLD, o recorte por complexidade e a distribuição por faixas de erro ajudam a priorizar grupos com maior divergência entre valores registrados e estimados.

As Figuras 23 e 24 exploram as camadas MUNKA_GOLD e MUNKA_ML com agregações próprias; por isso, seus KPIs de volume não são diretamente comparáveis aos 5.000 registros de `ML\_TAREFA\_FEATURES` usados no treinamento. A aplicação retrospectiva oficial da Seção 6.8 utiliza exatamente 100 tarefas do Snowflake.

No ambiente reproduzível, o Metabase é executado em contêiner Docker e conectado ao Snowflake conforme a configuração descrita no Anexo I.

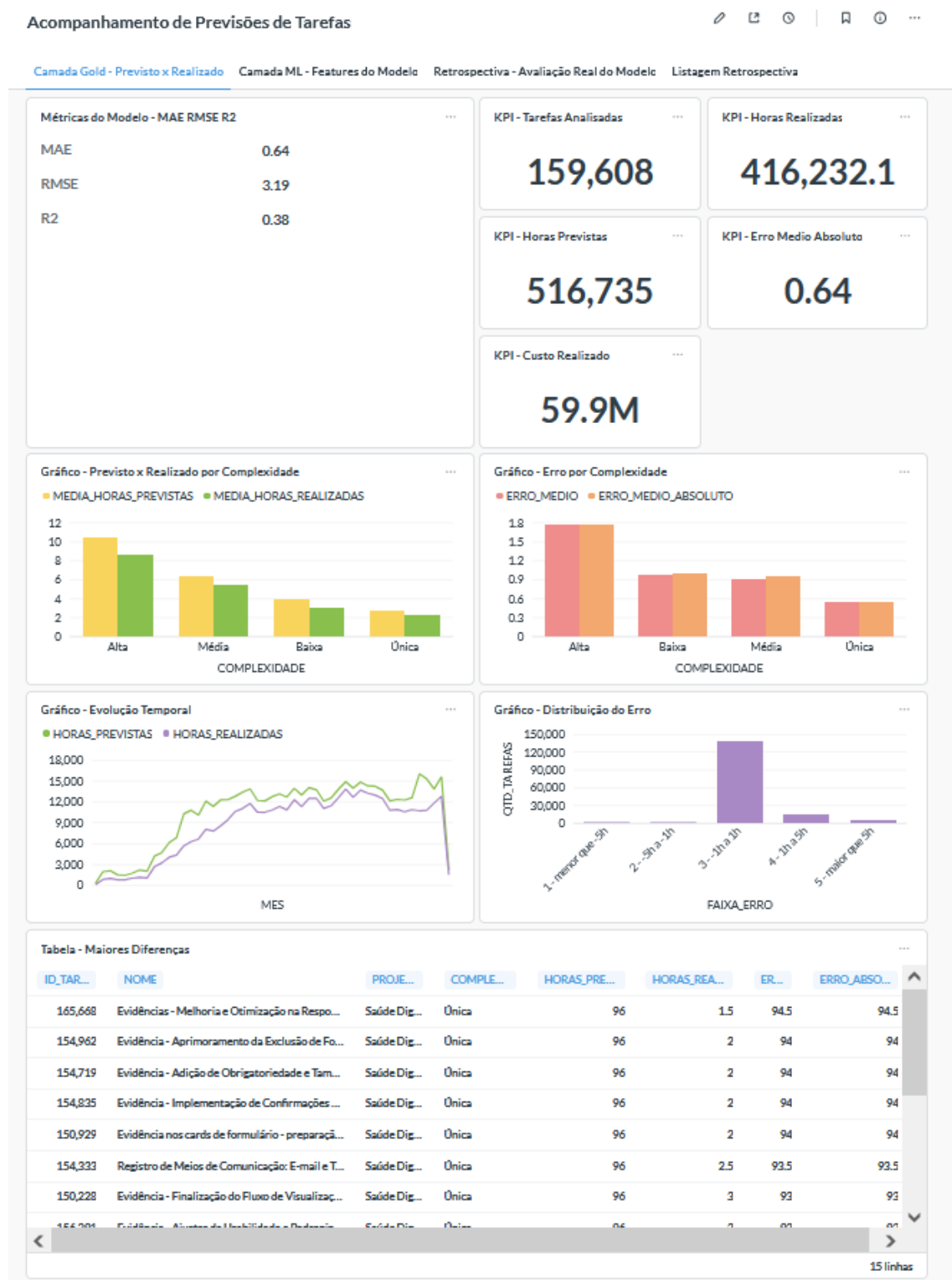


Figura 23. Dashboard GOLD com comparação entre Horas Previstas pelo ML e Horas Realizadas.

Acompanhamento de Previsões de Tarefas

✎ 📄 🕒 | 📌 ⌚ ...

Camada Gold - Previsto x Realizado Camada ML - Features do Modelo Retrospectiva - Avaliação Real do Modelo Listagem Retrospectiva



Figura 24. Dashboard ML - Features do Modelo.

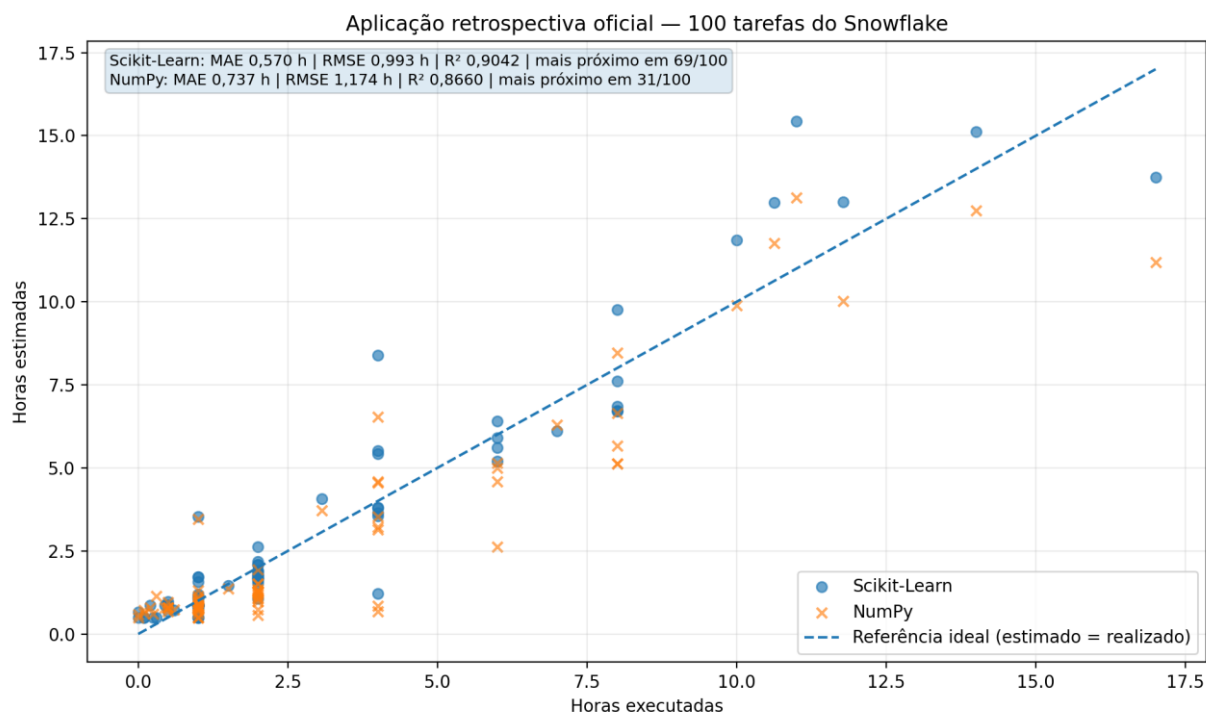


Figura 25. Comparação retrospectiva oficial entre horas registradas e horas estimadas (100 tarefas do Snowflake).

A Figura 25 foi gerada diretamente a partir da execução oficial documentada nos arquivos `analise\_retrospectiva.csv` e `analise\_retrospectiva\_metrics.json`, assegurando coerência entre a visualização e os valores apresentados na Tabela 5. As Figuras 23 e 24 registram a interface dos dashboards construídos no Metabase.

10. Reprodutibilidade da Solução

A solução procura manter suas etapas reproduzíveis por meio da combinação de versionamento de código, Docker, Airflow, dbt, CloudFormation e scripts Python.

A estrutura permite que as principais etapas sejam executadas de forma independente ou orquestrada.

O fluxo geral é composto por:

1. disponibilização dos arquivos de dados no Amazon S3;
2. criação das estruturas RAW no Snowflake;
3. carga dos dados para MUNKA_RAW;
4. execução dos modelos Staging;
5. execução das transformações Intermediate;

6. construção dos modelos Gold e ML;
7. execução dos testes de qualidade do dbt;
8. preparação do conjunto de dados para Machine Learning;
9. treinamento e avaliação dos modelos;
10. registro dos experimentos no MLflow;
11. execução da análise retrospectiva em lote e disponibilização das estruturas analíticas para consumo.

A modularização permite identificar de forma clara qual componente é responsável por cada etapa e reduz o acoplamento entre ingestão, transformação, treinamento e aplicação retrospectiva do modelo.

10.1. Avaliação do Pipeline

Além da avaliação do modelo, o pipeline foi analisado quanto à execução das etapas, organização dos dados, qualidade após transformação, rastreabilidade e limitações arquiteturais. A síntese abaixo consolida os principais elementos documentados no projeto.

Aspecto	Evidência no projeto	Avaliação
Ingestão	S3 → Snowflake RAW por COPY INTO e Airflow	Fluxo documentado e reproduzível
Organização	Schemas RAW, STG, INT, GOLD e ML	Separação clara de responsabilidades
Qualidade	78 testes dbt unique/not_null informados como aprovados	Validação automatizada antes do consumo
Transformação	~90 modelos dbt e engenharia de atributos no Snowflake	Processamento concentrado no DW
Rastreabilidade	15.420 registros RAW → 5.000 na camada ML → treino/avaliação	Linhas de transformação documentadas
ML/Aplicação	K-Fold, holdout, MLflow e análise retrospectiva em lote	Etapas separadas por finalidade

Tabela 8. Síntese da avaliação do pipeline.

Como limitações arquiteturais permanecem a dependência de componentes locais em Docker/MLflow, a ausência de comprovação de implantação de todos os recursos declarados no CloudFormation e a necessidade de manter os painéis analíticos sincronizados com cada execução oficial do pipeline.

11. Limitações da Solução

As principais limitações identificadas nesta primeira versão são:

Histórico Limitado em Algumas Categorias: Determinados tipos raros de tarefas no sistema legado possuem pouca amostragem histórica, aumentando a variância das estimativas nessas categorias específicas.

Dependência de Execução Local do SQLite no MLflow: No ambiente de desenvolvimento Docker/Windows, o SQLite do MLflow exige armazenamento no caminho do container (/tmp/mlflow.db) para evitar bloqueios de I/O em pastas compartilhadas.

Modelos Não-Lineares de Gradient Boosting: O escopo do trabalho focou estritamente em Redes Neurais MLP (Scikit-Learn e implementação customizada em NumPy), sem explorar algoritmos de árvores de decisão como XGBoost ou LightGBM.

Disponibilidade temporal das features: o modelo atual utiliza atributos extraídos das evidências de execução e, por esse motivo, possui finalidade retrospectiva. Ele não deve ser utilizado como modelo antecipado para tarefas ainda não iniciadas. A construção de um modelo preditivo antecipado exige um conjunto distinto de features disponíveis antes da execução.

Qualidade dos apontamentos de horas: a variável-alvo depende dos registros efetuados no sistema. Inconsistências, atrasos ou diferenças de prática no apontamento podem introduzir ruído tanto no treinamento quanto na interpretação dos desvios.

Generalização da aplicação retrospectiva em lote: as 100 tarefas utilizadas no Passo 6 foram obtidas do Snowflake para demonstração analítica e não constituem um novo holdout independente. Por isso, as métricas dessa execução não substituem a avaliação formal realizada no conjunto de teste isolado.

Risco de uso na primeira versão: as estimativas não devem ser utilizadas isoladamente para glosa, aceite, pagamento ou responsabilização. A solução funciona como instrumento de triagem e apoio à análise, devendo ser combinada com regras contratuais e avaliação técnica.

12. Propostas de Evolução do Trabalho

A solução desenvolvida neste trabalho estabelece uma arquitetura capaz de transformar os registros transacionais do MUNKA em informações analíticas e estimativas retrospectivas destinadas ao acompanhamento do esforço das tarefas. A arquitetura implementada organiza os dados nas camadas RAW, Staging, Intermediate, Gold e Machine Learning, permitindo incorporar novas fontes sem modificar conceitualmente o fluxo principal de processamento.

Embora os resultados obtidos indiquem capacidade relevante de explicação da variável-alvo no conjunto de avaliação utilizado, o próprio processo de avaliação deve ser interpretado considerando as características e limitações dos dados disponíveis.

Um dos principais caminhos para evolução da solução consiste, portanto, na **ampliação das fontes de dados utilizadas para caracterizar cada tarefa**.

Atualmente, grande parte das informações utilizadas pelo modelo é proveniente do próprio MUNKA, incluindo informações estruturadas das tarefas e atributos extraídos de suas evidências. Entretanto, o ciclo completo de desenvolvimento e manutenção de software produz informações em outros sistemas corporativos que podem apresentar forte relação com o esforço efetivamente necessário para execução de uma atividade.

Nesse contexto, duas fontes possuem especial potencial de enriquecimento do conjunto de dados: **o GitLab e o sistema de GLPI-Helpdesk**.

12.1. Ampliação das Fontes de Dados

A evolução proposta consiste em deixar de analisar a tarefa exclusivamente a partir das informações existentes no MUNKA e passar a observar todo o seu **ciclo de vida operacional e técnico**.

Conceitualmente, uma demanda poderia ser representada pelo seguinte fluxo:

Helpdesk → MUNKA → GitLab → execução técnica → evidências → medição

Cada uma dessas etapas apresenta uma visão diferente sobre a mesma atividade.

O Helpdesk representa principalmente a **origem e o contexto da demanda**. O MUNKA representa seu **planejamento, acompanhamento e medição**. O GitLab, por sua vez, registra parte relevante da **execução técnica necessária para implementação da solução**.

A integração dessas informações permitiria construir uma representação mais abrangente das tarefas utilizadas pelo modelo de Machine Learning.

12.2. Integração com o GitLab

Uma das principais evoluções propostas consiste na utilização do **GitLab como nova fonte de dados do pipeline**.

O próprio MUNKA já possui evidências que podem conter referências a Pull Requests e links para repositórios GitLab, demonstrando que existe uma relação natural entre as tarefas registradas no sistema e os artefatos produzidos durante sua execução.

A integração direta com o GitLab permitiria substituir parte das informações inferidas a partir de textos por informações estruturadas obtidas diretamente da plataforma de desenvolvimento.

Entre os elementos potencialmente coletados podem estar:

- projeto ou repositório relacionado à tarefa;
- branch associada à implementação;
- commits realizados;
- autores dos commits;
- datas e horários dos commits;
- intervalo entre primeiro e último commit;
- quantidade de commits;
- quantidade de arquivos modificados;
- quantidade de linhas adicionadas e removidas;
- tipos de arquivos alterados;
- linguagens ou tecnologias envolvidas;
- Merge Requests/Pull Requests relacionados;
- quantidade de revisões realizadas;
- comentários associados ao processo de revisão;
- quantidade de pipelines executados;
- pipelines com sucesso ou falha;
- número de tentativas necessárias até a conclusão;
- tags e releases relacionadas;
- rastreabilidade entre tarefa, branch, commit e Merge Request.

Essas informações podem gerar indicadores capazes de representar de maneira mais objetiva a **complexidade técnica real da implementação**.

Por exemplo, duas tarefas classificadas inicialmente com a mesma complexidade podem apresentar comportamentos técnicos bastante distintos. Uma pode exigir alteração em apenas um arquivo e poucos commits, enquanto outra pode envolver múltiplos repositórios, dezenas de arquivos, diversas revisões e várias execuções de pipeline.

Essas diferenças dificilmente são completamente representadas apenas pelos atributos cadastrais existentes na tarefa.

Assim, o GitLab poderá fornecer novas variáveis explicativas ao modelo, tais como:

quantidade de commits

quantidade de arquivos alterados

linhas adicionadas

linhas removidas

quantidade de Merge Requests

quantidade de revisões

quantidade de falhas de pipeline

quantidade de sistemas ou repositórios afetados

duração do ciclo de desenvolvimento

diversidade de tecnologias modificadas

Essas características poderão ser avaliadas quanto à sua correlação com o esforço efetivamente registrado.

12.3. Integração com o Sistema de Helpdesk

Outro caminho relevante de evolução consiste na incorporação das informações do sistema de **Helpdesk responsável pelo registro inicial das demandas e incidentes**.

Em muitos casos, a tarefa registrada no MUNKA pode ter origem em uma solicitação anterior realizada no Helpdesk. Essa solicitação contém informações produzidas antes mesmo da criação da tarefa técnica.

A integração dessas informações permitiria analisar elementos como:

- descrição original da solicitação;
- categoria do chamado;
- tipo da demanda;
- sistema afetado;
- prioridade;
- criticidade;
- solicitante;
- unidade solicitante;
- data e hora da abertura;
- quantidade de interações;
- quantidade de reatribuições;
- grupos técnicos envolvidos;
- tempo de atendimento;
- tempo até encaminhamento para desenvolvimento;

- anexos existentes;
- histórico textual do atendimento;
- reaberturas;
- incidentes anteriores semelhantes.

Essas informações podem ser particularmente importantes porque representam características conhecidas **antes da execução da tarefa**.

Essa distinção é especialmente relevante para a evolução futura do projeto. A implementação atual é retrospectiva e pode utilizar informações produzidas durante ou após a execução; um modelo preditivo antecipado, por outro lado, deverá utilizar exclusivamente dados disponíveis antes do início da atividade.

Para essa evolução, as variáveis deverão ser separadas formalmente em dois grupos:

- 1) Features disponíveis antes do início da execução, adequadas ao modelo preditivo antecipado;
- 2) Features produzidas durante ou após a execução da tarefa, mantidas no modelo retrospectivo e nas análises históricas.

Essa separação evitará o uso indevido de informações futuras no cenário antecipado e permitirá manter, ao mesmo tempo, o valor analítico das evidências no cenário retrospectivo.

12.4. Construção de uma Visão Integrada da Demanda

A integração das três fontes possibilitaria criar uma representação única da demanda ao longo de todo o seu ciclo de vida. A estrutura conceitual poderia ser:

A identificação de uma **chave de rastreabilidade entre essas fontes** será uma etapa fundamental dessa evolução.

Essa associação poderá utilizar, conforme a estrutura existente nos sistemas, elementos como:

```

ID do chamado Helpdesk
↓
ID da tarefa MUNKA
↓
Projeto GitLab
↓
Branch
↓
Merge Request
  
```



Commits

Dessa forma, será possível reconstruir a trajetória completa de uma demanda.

12.5. Novas Estratégias de Engenharia de Atributos

A incorporação dessas fontes permitirá ampliar significativamente a engenharia de atributos.

Atualmente, o projeto transforma conteúdos textuais e HTML das evidências em variáveis quantitativas e indicadores binários utilizando principalmente expressões regulares executadas no Snowflake.

Como evolução, poderão ser construídos atributos pertencentes a diferentes dimensões.

12.5.1. Complexidade da solicitação

Derivada das informações do Helpdesk:

Tipo da solicitação
prioridade
criticidade
quantidade de interações
quantidade de áreas envolvidas
tamanho da descrição
quantidade de anexos

12.5.2. Complexidade técnica

Derivada do GitLab:

quantidade de commits
arquivos modificados
linhas adicionadas/removidas
repositórios envolvidos
Merge Requests
pipelines executados
falhas de pipeline

12.5.3. Complexidade temporal

tempo entre abertura do chamado e criação da tarefa
tempo entre criação e primeiro commit
tempo entre primeiro e último commit
tempo de revisão
tempo entre Merge Request e merge
tempo total da demanda

12.5.4. Complexidade organizacional

número de desenvolvedores envolvidos
número de equipes envolvidas
número de revisores
quantidade de transferências do chamado
quantidade de responsáveis pela tarefa

A combinação dessas dimensões pode produzir uma caracterização significativamente mais rica da tarefa.

12.6. Evolução dos Modelos de Machine Learning

Com o aumento da quantidade e diversidade das features, também poderá ser realizada uma nova etapa de experimentação dos modelos.

A implementação atual utiliza redes MLP nas versões NumPy e Scikit-Learn, com otimização de hiperparâmetros utilizando Optuna e rastreamento dos experimentos por meio do MLflow.

Como evolução, poderá ser avaliado se outros algoritmos apresentam melhor capacidade de generalização sobre o conjunto enriquecido de dados.

Entre as estratégias possíveis estão:

- Random Forest;
- Gradient Boosting;
- XGBoost;
- LightGBM;
- CatBoost;
- modelos ensemble;
- redes neurais com novas arquiteturas.

O MLflow poderá continuar sendo utilizado para comparar essas abordagens, registrando os experimentos, parâmetros, métricas e artefatos, mantendo a rastreabilidade já estabelecida no projeto.

12.6.1. Modelo Preditivo Antecipado

Como evolução prioritária, propõe-se desenvolver um segundo modelo especificamente destinado à previsão antecipada do esforço de novas tarefas. Nesse cenário, será definido um instante de previsão anterior ao início da execução, e somente informações disponíveis até esse momento poderão compor a matriz de atributos.

Features derivadas de evidências produzidas durante a execução, como presença de commits, Pull Requests, blocos de código, anexos e outros elementos posteriores, não deverão participar do modelo antecipado. Essas informações continuarão disponíveis para o modelo retrospectivo e para análises históricas.

A integração com o Helpdesk é especialmente relevante para essa evolução, pois pode fornecer atributos conhecidos antes do desenvolvimento, como categoria, prioridade, criticidade, sistema afetado, descrição inicial e unidade solicitante. O desempenho desse futuro modelo deverá ser avaliado separadamente, preferencialmente também com validação temporal.

12.7. Processamento Semântico dos Textos

Outro caminho de evolução consiste na substituição ou complementação da atual engenharia de atributos baseada em expressões regulares por técnicas efetivas de **Processamento de Linguagem Natural**.

O próprio relatório destaca que a implementação atual ainda não utiliza modelos linguísticos ou embeddings, realizando principalmente transformação do texto em indicadores estruturados.

Os textos provenientes de:

Helpdesk + MUNKA + evidências + GitLab

poderiam futuramente ser processados para obtenção de embeddings ou representações semânticas.

Isso permitiria identificar, por exemplo, tarefas historicamente semelhantes mesmo quando descritas com palavras diferentes.

Uma possível aplicação seria:

Nova solicitação



representação semântica



busca por tarefas semelhantes



tarefas históricas



horas executadas anteriormente



feature adicional para o modelo

Dessa forma, o sistema poderia combinar uma abordagem preditiva com uma abordagem baseada em similaridade histórica.

12.8. Monitoramento Contínuo e Aprendizado do Modelo

Outra evolução possível é transformar o pipeline atual em um processo de **aprendizado contínuo**.

Como o Airflow já coordena as etapas de ingestão, transformação, testes, treinamento e análise retrospectiva em lote, existe uma base arquitetural para automatizar novos ciclos de treinamento.

Novas tarefas concluídas poderiam periodicamente ser incorporadas ao conjunto histórico.

O processo poderia seguir:

Novas tarefas concluídas



Atualização MUNKA / GitLab / Helpdesk



Pipeline ELT



Construção das features



Avaliação de Data Drift



Novo treinamento



Comparação com modelo atual



MLflow



Promoção do modelo somente se houver melhoria

Essa estratégia permitiria acompanhar se o comportamento das tarefas muda ao longo do tempo e se o modelo continua apresentando desempenho adequado.

12.9. Governança, Segurança e Qualidade dos Dados

A integração com GitLab e Helpdesk também exigirá ampliação dos mecanismos de governança existentes.

Além dos testes `unique` e `not_null` já empregados nos modelos dbt, poderão ser incorporados testes específicos de integridade entre as diferentes fontes.

Por exemplo:

chamado Helpdesk → tarefa MUNKA

tarefa MUNKA → projeto GitLab

tarefa MUNKA → branch

branch → commits

Merge Request → tarefa

Também deverão ser estabelecidas políticas relacionadas a:

- autenticação nas APIs;
- proteção de tokens de acesso;
- controle de permissões;
- anonimização ou exclusão de informações desnecessárias;
- definição de dados que poderão ou não ser utilizados no treinamento;
- rastreabilidade da origem das features;
- versionamento dos datasets utilizados nos experimentos.

12.10. Impactos Esperados

A principal hipótese associada à evolução proposta é que a utilização conjunta dos dados do **MUNKA**, **GitLab** e **Helpdesk** possa ampliar a capacidade do modelo de caracterizar corretamente uma tarefa.

O modelo deixaria de analisar somente aquilo que foi registrado administrativamente sobre a demanda e passaria a considerar três perspectivas complementares:

contexto da solicitação → Helpdesk

gestão e medição da tarefa → MUNKA

execução técnica → GitLab

A arquitetura passaria conceitualmente para:

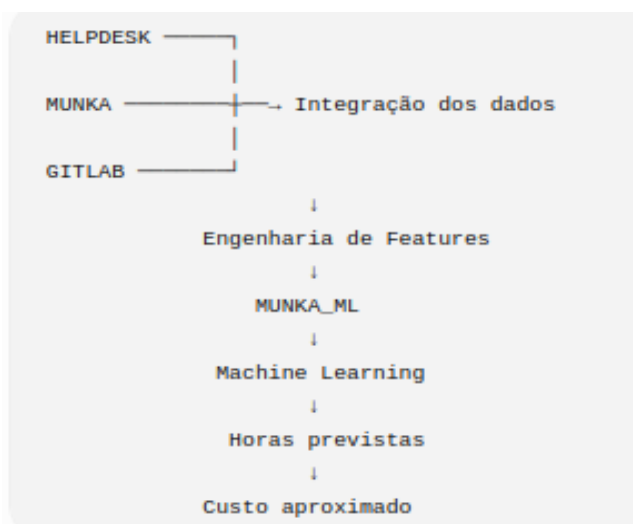


Figura 26. Junção de fontes de dados diversas.

Portanto, espera-se que o enriquecimento dos dados possibilite reduzir MAE, MSE e RMSE e aumentar a capacidade de generalização do modelo, especialmente em tarefas cuja complexidade não pode ser adequadamente representada apenas pelas informações existentes no MUNKA.

13. Conclusão

A solução desenvolvida transforma os dados originalmente registrados no MUNKA em uma estrutura analítica e retrospectiva por meio de um pipeline ELT. **Esta primeira versão visou obter a retrospectiva das horas de execução das tarefas e obter dados que facilitem a revisão e validação das tarefas no momento de fiscalização das mesmas.**

O Amazon S3 funcionou como ponto de armazenamento dos arquivos de origem, enquanto o Snowflake concentrou o armazenamento analítico e o processamento das transformações.

O dbt organizou os dados em diferentes níveis de tratamento, desde a camada RAW até a construção de fatos, dimensões e tabelas específicas para Machine Learning. O Apache Airflow coordenou a execução dessas etapas e estabeleceu suas dependências.

As informações textuais existentes nas evidências das tarefas foram convertidas em atributos estruturados por meio de regras e expressões regulares executadas diretamente no Snowflake.

A camada de Machine Learning utilizou esses dados para treinamento de modelos MLP voltados à estimativa retrospectiva do esforço das tarefas em horas, comparando uma implementação manual em NumPy com uma implementação baseada no Scikit-Learn.

O MLflow registrou os experimentos e seus resultados, enquanto o Optuna foi utilizado na experimentação com diferentes configurações de hiperparâmetros. Na avaliação final registrada, o modelo Scikit-Learn apresentou desempenho superior.

Dessa forma, a solução integra Engenharia de Dados, Cloud Computing e Aprendizagem de Máquina em um único fluxo, estabelecendo a ligação entre os registros transacionais do MUNKA, a estimativa retrospectiva de esforço e a identificação de desvios para apoio à análise das tarefas.

14. Referências

AMAZON WEB SERVICES. **Amazon Simple Storage Service (Amazon S3) – User Guide**. [S. l.]: Amazon Web Services, s.d. Disponível em: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>. Acesso em: 17 ago. 2026.

AMAZON WEB SERVICES. **AWS CloudFormation Documentation**. [S. l.]: Amazon Web Services, s.d. Disponível em: <https://docs.aws.amazon.com/cloudformation/>. Acesso em: 17 ago. 2026.

AMAZON WEB SERVICES. **AWS::S3::Bucket – AWS CloudFormation Template Reference**. [S. l.]: Amazon Web Services, s.d. Disponível em: <https://docs.aws.amazon.com/AWSCloudFormation/latest/TemplateReference/aws-resource-s3-bucket.html>. Acesso em: 17 ago. 2026.

APACHE SOFTWARE FOUNDATION. **Apache Airflow Documentation: Core Concepts**. [S. l.]: Apache Software Foundation, s.d. Disponível em: <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/index.html>. Acesso em: 17 ago. 2026.

DBT LABS. **dbt Developer Hub: What is dbt?** [S. l.]: dbt Labs, s.d. Disponível em: <https://docs.getdbt.com/docs/introduction>. Acesso em: 17 ago. 2026.

METABASE. **Metabase Documentation**. [S. l.]: Metabase, s.d. Disponível em: <https://www.metabase.com/docs/latest/>. Acesso em: 17 ago. 2026.

METABASE. **Introduction to dashboards**. [S. l.]: Metabase, s.d. Disponível em: <https://www.metabase.com/docs/latest/dashboards/introduction>. Acesso em: 17 ago. 2026.

MLFLOW PROJECT. **MLflow Documentation**. [S. l.]: MLflow Project, s.d. Disponível em: <https://mlflow.org/docs/latest/>. Acesso em: 17 ago. 2026.

MLFLOW PROJECT. **MLflow Tracking**. [S. l.]: MLflow Project, s.d. Disponível em: <https://mlflow.org/docs/latest/ml/tracking/>. Acesso em: 17 ago. 2026.

NUMPY DEVELOPERS. **NumPy Documentation**. [S. l.]: NumPy Developers, s.d. Disponível em: <https://numpy.org/doc/stable/>. Acesso em: 17 ago. 2026.

OPTUNA. **Optuna Documentation**. [S. l.]: Optuna, s.d. Disponível em: <https://optuna.readthedocs.io/>. Acesso em: 17 ago. 2026.

PYTHON SOFTWARE FOUNDATION. **Python Documentation**. [S. l.]: Python Software Foundation, s.d. Disponível em: <https://docs.python.org/>. Acesso em: 17 ago. 2026.

SCIKIT-LEARN DEVELOPERS. **MLPRegressor: Multi-layer Perceptron regressor**. [S. l.]: Scikit-learn, s.d. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html. Acesso em: 17 ago. 2026.

SCIKIT-LEARN DEVELOPERS. **Neural network models (supervised)**. [S. l.]: Scikit-learn, s.d. Disponível em: https://scikit-learn.org/stable/modules/neural_networks_supervised.html. Acesso em: 17 ago. 2026.

SNOWFLAKE. **Snowflake Documentation**. [S. l.]: Snowflake, s.d. Disponível em: <https://docs.snowflake.com/en/>. Acesso em: 17 ago. 2026.

SNOWFLAKE. **Snowflake Key Concepts and Architecture**. [S. l.]: Snowflake, s.d. Disponível em: <https://docs.snowflake.com/en/user-guide/intro-key-concepts>. Acesso em: 17 ago. 2026.

ANEXO I: Implantação da Aplicação – Projeto Munka.

Repositório da Aplicação: <https://github.com/ivaniojr/dbt-snowflake-airflow-main.git>

Guia técnico completo de implantação, provisionamento e configuração da infraestrutura moderna de dados e Machine Learning do **Projeto MUNKA (IFG)**.

Este anexo consolida parâmetros de configuração, referências de autenticação, procedimentos de provisionamento, orquestração no Apache Airflow, transformações no dbt Core, experimentos no MLflow e visualizações no Metabase. Valores secretos e chaves privadas não devem ser armazenados no documento nem versionados no repositório.



Sumário

1. [Visão Geral da Arquitetura e Componentes](#)
2. [Pré-Requisitos e Softwares Necessários](#)
3. [Matriz Centralizada de Credenciais e Variáveis \(.env\)](#)
4. [Provisionamento e Configuração na Nuvem AWS](#)
5. [Configuração e Segurança no Snowflake](#)
6. [Implantação da Stack Local com Docker e Airflow](#)
7. [Execução e Validação do Pipeline de Dados \(dbt Core\)](#)
8. [Ambiente de Machine Learning, HPO e MLflow](#)
9. [Configuração do Metabase e Conexão com Snowflake](#)
10. [Checklist de Homologação e Troubleshooting](#)

1. Visão Geral da Arquitetura e Componentes

A solução adota a **Arquitetura Medallion** em nuvem com orquestração desacoplada e containerizada:

2. Pré-Requisitos e Softwares Necessários

Software / Recurso	Versão Recomendada	Finalidade
Git	>= 2.40	Clonagem e controle de versão do repositório

Software / Recurso	Versão Recomendada	Finalidade
Docker Desktop	>= 24.x (com Docker Compose v2)	Execução containerizada do Airflow, Postgres, Redis e Metabase
Python	3.10 a 3.13	Execução local de scripts, dbt e pipelines de ML
AWS CLI	>= 2.15 (opcional)	Provisionamento via CloudFormation e upload para o S3
Conta Snowflake	Standard ou superior	Data Warehouse em nuvem
Conta AWS	Acesso IAM e S3	Armazenamento de dados no Data Lake

3. Matriz Centralizada de Credenciais e Variáveis (.env)

O projeto adota o princípio de **Fonte Única da Verdade**. Todas as configurações são lidas do arquivo .env na raiz do projeto (gerado a partir de [credentials_template.env](#)):

```
cp credentials_template.env .env
```

3.1. Variáveis do Snowflake

Variável	Valor Padrão / Exemplo	Descrição
DBT_SNOWFLAKE_ACCOUNT	sfedu02-gfb24387	Identificador da conta Snowflake (Organization-Account)
DBT_SNOWFLAKE_USER	DRAGON	Nome do usuário técnico no Snowflake
DBT_SNOWFLAKE_DATABASE	DRAGON_DB	Banco de dados central
DBT_SNOWFLAKE_WAREHOUSE	DRAGON_WH	Virtual Warehouse para processamento
DBT_SNOWFLAKE_ROLE	TRAINING_ROLE	Role de execução com privilégios adequados
DBT_SNOWFLAKE_SCHEMA	MUNKA_RAW	Schema padrão para conexões dbt e RAW

	Especialização em Inteligência Artificial Aplicada
	Projeto Final Integrado – Módulo 2 Alunos: Ivânio José da Rocha Júnior e Robson Pereira da Silva

Variável	Valor Padrão / Exemplo	Descrição
DBT_SNOWFLAKE_PRIVATE_KEY_PATH	/opt/airflow/dags/rsa_key.p8	Caminho da chave privada PKCS#8 para autenticação RSA

3.2. Variáveis da AWS

Variável	Valor Padrão / Exemplo	Descrição
AWS_ACCESS_KEY_ID	<AWS_KEY_ID>	Chave de acesso do usuário IAM com permissão no S3
AWS_SECRET_ACCESS_KEY	<AWS_SECRET_KEY>	Chave secreta de autenticação na AWS
AWS_DEFAULT_REGION	us-east-2	Região AWS do Bucket S3 (ex: Ohio)
S3_BUCKET_NAME	munka-dev-070980587239-us-east-2	Nome globalmente único do bucket S3

3.3. Variáveis do Apache Airflow & Docker

Variável	Valor Padrão	Descrição
AIRFLOW_UID	50000	UID do usuário no container Linux
_AIRFLOW_WWW_USER_USERNAME	airflow	Usuário administrador do Airflow Webserver
_AIRFLOW_WWW_USER_PASSWORD	airflow	Senha de acesso ao Airflow Webserver
AIRFLOW_CUSTOM_IMAGE_NAME	airflow-dbt-snowflake:2.10.0	Imagem Docker customizada com dbt e providers instalados

3.4. Variáveis do MLflow

Variável	Valor Padrão	Descrição
MLFLOW_TRACKING_URI	sqlite:///opt/airflow/project/src/ml/mlflow.db	URI do banco SQLite compartilhado entre Airflow e ambiente local

4. Provisionamento e Configuração na Nuvem AWS

4.1. Opção Automatizada via CloudFormation (IaC)

O arquivo `infrastructure/cloudformation.yaml` define declarativamente os recursos AWS previstos e pode ser utilizado para provisionamento automatizado. A existência do template não comprova, por si só, a execução da stack; essa comprovação requer evidência do status da implantação, como `CREATE\_COMPLETE` no

CloudFormation. [infrastructure/cloudformation.yaml](#)

```
aws cloudformation create-stack \
```

```
--stack-name munka-data-infrastructure-dev \
```

```
--template-body file://infrastructure/cloudformation.yaml \
```

```
--parameters ParameterKey=EnvironmentName,ParameterValue=dev \
```

```
ParameterKey=S3BucketName,ParameterValue=munka-dev-070980587239-us-east-2 \
```

```
--capabilities CAPABILITY_NAMED_IAM \
```

```
--region us-east-2
```

4.2. Estrutura de Arquivos no Amazon S3

Os arquivos .csv extraídos do sistema legado MUNKA devem ser carregados na raiz do bucket **obrigatoriamente em letras minúsculas**:

```
s3://munka-dev-070980587239-us-east-2/
```

```
├─ ab_user.csv
```

```
├─ ab_project.csv
```

```
├─ ab_task.csv
```

```
├─ ab_task_log.csv
```

```
├─ ab_sprint.csv
```

```
├─ ab_custom_field.csv
```

```
└─ ... (demais 41 arquivos CSV exportados)
```

⚠ Atenção: O comando COPY INTO do Snowflake é estrito quanto ao *case-sensitive* dos arquivos CSV.

4.3. Política IAM de Menor Privilégio (Least Privilege)

Caso crie o usuário IAM manualmente pelo Console AWS, utilize a política abaixo:

```
{
  "Version": "2012-10-17",
  "Statement": [
```



```
{
  "Sid": "MunkaS3BucketAccess",
  "Effect": "Allow",
  "Action": [
    "s3:GetObject",
    "s3:GetObjectVersion",
    "s3:ListBucket",
    "s3:GetBucketLocation"
  ],
  "Resource": [
    "arn:aws:s3:::munka-dev-070980587239-us-east-2",
    "arn:aws:s3:::munka-dev-070980587239-us-east-2/*"
  ]
}
```

5. Configuração e Segurança no Snowflake

5.1. Setup de Banco, Schemas, Warehouse e Role

Execute os comandos SQL abaixo no Snowflake (como ACCOUNTADMIN ou SYSADMIN):

Os comandos a seguir refletem a configuração de desenvolvimento/homologação documentada no projeto e concedem privilégios amplos à `TRAINING\_ROLE`. Em ambiente de produção, recomenda-se reduzir os privilégios ao conjunto estritamente necessário para cada componente, aplicando o princípio de menor privilégio.

-- 1. Criar o Banco de Dados e Warehouse

```
CREATE DATABASE IF NOT EXISTS DRAGON_DB;
```

```
CREATE WAREHOUSE IF NOT EXISTS DRAGON_WH
```

```
WITH WAREHOUSE_SIZE = 'XSMALL'
```

```
AUTO_SUSPEND = 120
```

```
AUTO_RESUME = TRUE
```

```
INITIALLY_SUSPENDED = TRUE;
```

-- 2. Criar os Schemas da Arquitetura Medallion

```
USE DATABASE DRAGON_DB;
```

```
CREATE SCHEMA IF NOT EXISTS MUNKA_RAW;
```

```
CREATE SCHEMA IF NOT EXISTS MUNKA_STG;
```

```
CREATE SCHEMA IF NOT EXISTS MUNKA_INT;
```

```
CREATE SCHEMA IF NOT EXISTS MUNKA_GOLD;
```

```
CREATE SCHEMA IF NOT EXISTS MUNKA_ML;
```

-- 3. Criar a Role de Treinamento/Engenharia

```
CREATE ROLE IF NOT EXISTS TRAINING_ROLE;
```

```
GRANT USAGE ON WAREHOUSE DRAGON_WH TO ROLE TRAINING_ROLE;
```

```
GRANT ALL PRIVILEGES ON DATABASE DRAGON_DB TO ROLE TRAINING_ROLE;
```

```
GRANT ALL PRIVILEGES ON ALL SCHEMAS IN DATABASE DRAGON_DB TO ROLE  
TRAINING_ROLE;
```

```
GRANT ALL PRIVILEGES ON FUTURE SCHEMAS IN DATABASE DRAGON_DB TO ROLE  
TRAINING_ROLE;
```

```
GRANT ALL PRIVILEGES ON ALL TABLES IN DATABASE DRAGON_DB TO ROLE  
TRAINING_ROLE;
```

```
GRANT ALL PRIVILEGES ON FUTURE TABLES IN DATABASE DRAGON_DB TO ROLE  
TRAINING_ROLE;
```

```
GRANT ALL PRIVILEGES ON ALL VIEWS IN DATABASE DRAGON_DB TO ROLE  
TRAINING_ROLE;
```

```
GRANT ALL PRIVILEGES ON FUTURE VIEWS IN DATABASE DRAGON_DB TO ROLE  
TRAINING_ROLE;
```

5.2. Autenticação por Par de Chaves RSA (Key-Pair Authentication)

Para dispensar o uso de senhas e garantir segurança empresarial:

1. **Geração das chaves: a chave privada deve ser criada localmente, mantida fora do controle de versão e disponibilizada aos contêineres por volume ou caminho configurado em variável de ambiente. O arquivo `rsa\_key.p8` deve constar no `.gitignore`.**
2. # Gerar chave privada PKCS#8 sem senha
3. `openssl genrsa 2048 | openssl pkcs8 -topk8 -inform PEM -out rsa_key.p8 -nocrypt`

4.

5. # Extrair a chave pública

`openssl rsa -in rsa_key.p8 -pubout -out rsa_key.pub`

6. **Associar a Chave Pública ao Usuário no Snowflake:**

7. `ALTER USER DRAGON SET RSA_PUBLIC_KEY = 'MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA...';`

8. `GRANT ROLE TRAINING_ROLE TO USER DRAGON;`

9. `ALTER USER DRAGON SET DEFAULT_ROLE = TRAINING_ROLE;`

`ALTER USER DRAGON SET DEFAULT_WAREHOUSE = DRAGON_WH;`

6. Implantação da Stack Local com Docker e Airflow

6.1. Inicialização dos Contêineres

Na pasta [airflow/](#):

`cd airflow`

1. Construir e inicializar todos os serviços em background

`docker compose up -d --build`

2. Verificar o status de saúde dos containers

`docker ps`

Serviços Ativos e Portas:

- **Airflow Webserver:** `http://localhost:8081` (Usuário: airflow / Senha: airflow)
- **MLflow UI:** `http://localhost:5000`
- **Metabase BI:** `http://localhost:3000`
- **Postgres (Metadados Airflow):** Porta 5432
- **Redis (Celery Broker):** Porta 6379

6.2. Estrutura do Pipeline Orquestrado (7 Passos + DAG Master)

O Airflow gerencia a execução sequencial ponta a ponta através da DAG Master

[airflow/dags/dag_munka_full_pipeline.py](#):

1. **dag_munka_full_pipeline:** DAG Master orquestradora geral (dispara e monitora os passos 1 a 7).

2. **passo1_munka_dbt_create_raw_tables:** Criação DDL e estrutura na camada MUNKA_RAW.
3. **passo2_s3_to_snowflake_munka_raw:** Ingestão dos CSVs do S3 para o Snowflake via COPY INTO.
4. **passo3_munka_dbt_create_stg:** Execução do dbt Staging (limpeza, tipagem e deduplicação via QUALIFY).
5. **passo4_munka_dbt_run_marts:** Execução do dbt Marts (Gold + ML) e 78 testes de integridade de dados.
6. **passo5_ml_hpo_e_retreinamento:** Otimização HPO (Optuna), retreinamento 5-Fold Cross Validation e registro de runs/artefatos no MLflow.
7. **passo6_batch_inference:** Análise retrospectiva em lote com os modelos selecionados (``batch_inference.py``), gerando ``analise_retrospectiva.csv`` e ``analise_retrospectiva_metrics.json`` e registrando a execução no MLflow (``MUNKA_ML_Analise_Retrospectiva``).
8. **passo7_ml_carga_analise_retrospectiva:** Sincronização do CSV para o diretório de seeds, execução do dbt seed e materialização da mart MUNKA_ML.ML_ANALISE_RETROSPECTIVA no Snowflake.

7. Execução e Validação do Pipeline de Dados (dbt Core)

Para executar o dbt diretamente no terminal local:

1. Ativar o ambiente virtual

```
python -m venv venv
```

```
venv\Scripts\activate      # Windows
```

```
# source venv/bin/activate  # Linux/macOS
```

2. Instalar dependencias

```
pip install -r requirements.txt
```

3. Testar a conexão com o Snowflake

```
cd src/dbt
```

```
dbt debug --profiles-dir .
```

4. Compilar os 90 modelos

dbt compile --profiles-dir .

5. Executar o pipeline de transformacao completo

dbt run --profiles-dir .

6. Executar a suite de testes de integridade (unique e not_null)

dbt test --profiles-dir .

8. Ambiente de Machine Learning, HPO e MLflow

8.1. Iniciar o Servidor MLflow UI

O banco de rastreamento fica centralizado no arquivo [src/ml/mlflow.db](#). Para iniciar a interface visual do MLflow:

A partir da raiz do projeto:

mlflow ui --backend-store-uri sqlite:///src/ml/mlflow.db --port 5000

Ou a partir da pasta src/ml:

cd src/ml

mlflow ui --backend-store-uri sqlite:///mlflow.db --port 5000

Acesse no navegador: **http://localhost:5000**

8.2. Scripts do Módulo de Machine Learning

Script	Função Principal	Experimento MLflow
src/ml/train.py	Treinamento Baseline & Retrospectivo comparando NumPy vs Sklearn (5-Fold CV + Holdout)	MUNKA_MLP_Retrospective
src/ml/hpo.py	Otimização Automática de Hiperparâmetro s via Optuna (Learning Rate,	Auditoria_MLP_HPO

Script	Função Principal	Experimento MLflow
	Camadas, Neurônios, Alpha)	
src/ml/train_best.py	Retreinamento final com os melhores hiperparâmetros	Auditoria_MLP_Best_Params e exportação dos modelos campeões
src/ml/batch_inference.py	Análise Retrospectiva em lote sobre tarefas executadas (Passo 6), gerando CSV e métricas	MUNKA_ML_Analise_Retrospectiva
src/ml/load_analise_retrospectiva.py	Carga direta e autônoma do CSV de auditoria -- no Snowflake via conector Python	
src/ml/export_evaluation_dataset.py	Exportação de dataset de teste Holdout isolado para homologação auditável	--

8.3. Experimentos Registrados no MLflow

8.4. Resultados Oficiais do Benchmark de Modelos

Desempenho no Retreinamento Final (5-Fold Cross Validation):

Modelo	Arquitetura / Topologia	MSE Val.	R ²	Status
MLP Scikit-Learn (Campeão)	2 camadas (64, 128), $lr = 0.0033$, $\alpha = 0.000025$	5.46	0.75 (75%)	Modelo selecionado

	Especialização em Inteligência Artificial Aplicada
	Projeto Final Integrado – Módulo 2 Alunos: Ivânio José da Rocha Júnior e Robson Pereira da Silva

Modelo	Arquitetura / Topologia	MSE Val.	R ²	Status
MLP Sklearn Restrito	2 camadas (64, 32), $lr = 0.0027$	5.49	0.74 (74%)	Controle comparativo
MLP NumPy (Puro)	2 camadas (16, 32), $lr = 0.0375$	6.90	0.68 (68%)	Implementação manual
Baseline Linear	Regressão Linear Simples	345.12	0.58 (58%)	Baseline

Desempenho na Análise Retrospectiva em Lote (análise_retrospectiva_metrics.json):

Os valores abaixo correspondem à execução oficial de 14/08/2026, com 100 tarefas provenientes do Snowflake e `feature\_set\_name = retrospective\_features\_v1`, conforme os artefatos `análise\_retrospectiva.csv` e `análise\_retrospectiva\_metrics.json`.

Métrica	Scikit-Learn	NumPy	MLP
Erro Médio Absoluto (MAE)	0,5700 h	0,7365 h	
Raiz do Erro Quadrático Médio (RMSE)	0,9929 h	1,1743 h	
Erro Quadrático Médio (MSE)	0,9859	1,3790	
Coeficiente de Determinação (R²)	0,9042 (90,4%)	0,8660 (86,6%)	

9. Configuração do Metabase e Conexão com Snowflake

O Metabase consome diretamente as tabelas dimensionais da camada Gold e os resultados analíticos da camada ML.

1. Acesse <http://localhost:3000>.
2. Vá em **Admin > Databases > Add a database**.
3. Preencha o formulário exatamente com as credenciais abaixo:

Campo no Formulário	Valor
Database type	Snowflake
Connection string (optional)	(em branco)
Display name	Snowflake - MUNKA
Account name	sfedu02-gfb24387
Username	DRAGON
Authenticate with user and password	Desligado (usar chave RSA)

Campo no Formulário	Valor
RSA private key (PKCS8/.p8)	Local file path
File path	/metabase-data/rsa_key.p8 (<i>caminho montado no container</i>)
Warehouse	DRAGON_WH
Database name	DRAGON_DB
Schemas	MUNKA_GOLD,MUNKA_ML (<i>ou All</i>)
Role (optional)	TRAINING_ROLE
Additional JDBC options (Advanced)	disablePlatformDetection=true

💡 **Dica de Troubleshooting JDBC:** A opção `disablePlatformDetection=true` é essencial para evitar o erro *Timed out after 10.0s* causado pela tentativa do driver JDBC de consultar metadados de nuvem dentro do Docker local.

10. Checklist de Homologação e Troubleshooting

Antes de liberar a aplicação para homologação final, valide os seguintes pontos:

- Arquivo .env configurado e preenchido na raiz do projeto.
- Bucket S3 criado na AWS com todos os 47 arquivos .csv em letras minúsculas.
- Chave RSA associada ao usuário DRAGON no Snowflake.
- Containers Docker ativos (docker ps) e saudáveis (airflow-webserver, scheduler, worker, metabase, postgres, redis).
- Zero erros de importação no Airflow (docker exec airflow-airflow-scheduler-1 airflow dags list-import-errors).
- DAG Master dag_munka_full_pipeline executada com sucesso de ponta a ponta (Passos 1 a 7 com status verde).
- 78 testes de integridade do dbt aprovados (dbt test).
- Experimentos e artefatos visualizáveis na interface do MLflow (<http://localhost:5000>).
- Metabase conectado ao Snowflake (<http://localhost:3000>) exibindo tabelas de MUNKA_GOLD e MUNKA_ML.